

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет об исследовательском проекте на тему:
Автономная навигация мобильного робота с использованием Robotic Operating
System: алгоритмы планирования и управления

Выполнил студент:

группы #БПМИ244, 2 курса

Злобин Герман Эдуардович

Принял руководитель проекта:

Дергачев Степан Алексеевич

Штатный преподаватель

Базовая кафедра Федерального исследовательского центра
«Информатика и управление» Российской академии наук

Содержание

1	Введение	5
2	Обзор литературы	7
2.1	Обзор методов глобального планирования	7
2.1.1	Алгоритмы поиска по графу	7
2.1.2	Алгоритмы случайной выборки (Sampling-based)	8
2.1.3	Интеллектуальные бионические алгоритмы	9
2.2	Обзор методов локального планирования	10
2.2.1	Классические контроллеры	10
2.2.2	Геометрические или реактивные методы	11
2.2.3	Оптимизационные или предсказательные методы	11
2.3	Сравнительный анализ и выбор алгоритмов	13
2.3.1	Выбор алгоритмов	15
2.4	Выводы по главе	16
3	Алгоритмы глобального планирования	17
3.1	Общие концепции алгоритмов A* и D* Lite	17
3.2	Модель и условия эксперимента	18
3.3	Результаты экспериментов	19
3.4	Анализ производительности в статической среде	20
3.5	Анализ производительности в динамической среде	20
3.6	Выводы по главе	21
4	Алгоритмы локального планирования	23
4.1	Общие концепции алгоритмов MPPI и RA-MPPI	23
4.2	Учёт прогноза подвижных препятствий	26
4.3	Модель и условия эксперимента	26
4.4	Результаты экспериментов	28
4.5	Анализ производительности в статической среде	30
4.6	Анализ производительности в динамической среде	31
4.7	Выводы по главе	32
5	Реализация навигационного стека в ROS 2 и сквозное тестирование	35
5.1	Архитектура и используемые компоненты	35
5.2	Разработанные программные пакеты	35
5.2.1	Реализация D* Lite в виде плагина Nav2	36
5.2.2	Реализация RA-MPPI в виде плагина Nav2	36
5.3	Среда тестирования и сценарии	37
5.4	Линейное предсказание траекторий динамических препятствий	38
5.5	Результаты экспериментов	39

5.6 Выводы по главе	41
Описание применения генеративной модели	42
Заключение	43
Список литературы	44

Аннотация

Работа посвящена исследованию методов планирования и управления в рамках задачи автономной навигации роботов в динамической среде. Задача рассмотрена на двух уровнях – глобального планирования и локального управления.

Выполнен систематический обзор графовых, сэмплирующих и бионических методов глобального планирования, а также классических, геометрических и оптимизационных методов локального управления. На основе сравнительного анализа для последующего экспериментального исследования выбраны инкрементальный планировщик D^* Lite и стохастический предсказательный контроллер RA-MPPI, в качестве базовых аналогов – A^* и MPPI.

Проведён сравнительный анализ на собственной программной модели: для глобальных планировщиков – 1000 независимых задач на каждую конфигурацию, для локальных контроллеров – 100 задач при разной плотности динамики. В ходе экспериментов выявлено, что A^* эффективен при разовом построении пути, тогда как D^* Lite сокращает время обновления решения при интенсивных изменениях карты и обеспечивает более предсказуемое время отклика. RA-MPPI снижает долю опасных сближений с препятствиями в несколько раз по сравнению с MPPI ценой пропорционального увеличения вычислительных затрат; явный прогноз подвижных объектов даёт наиболее выраженный выигрыш в сочетании с RA-MPPI при средней плотности динамики.

Практическим результатом являются два оригинальных программных пакета для ROS 2, интегрирующих выбранные алгоритмы в промышленный навигационный стек Nav2. Работоспособность стека подтверждена сквозным тестированием на симуляторе Gazebo Sim с моделью робота TurtleBot3 Burger по 16 конфигурациям сред и планировщиков. Наиболее устойчивой связкой при высокой плотности подвижных препятствий оказалась пара D^* Lite + MPPI, сочетающая быстрое инкрементальное перепланирование с минимальным числом принудительных торможений; пара D^* Lite + RA-MPPI лидирует по скорости в статических и малодинамичных сценариях. Полученные результаты подтверждают применимость предложенного подхода в составе промышленного навигационного стека.

Ключевые слова

Навигация; планирование пути; динамическая среда; мобильные роботы; D^* Lite; A^* ; MPPI; RA-MPPI; локальный и глобальный планировщик.

1 Введение

Современная мобильная робототехника активно внедряется в логистику, складскую автоматизацию, производство и городскую инфраструктуру, где ключевой задачей становится обеспечение автономной навигации. Способность робота безопасно и эффективно прокладывать маршрут в условиях, заранее неизвестных или меняющихся во времени, напрямую определяет его применимость в реальных приложениях.

Реальные среды эксплуатации мобильных роботов – склады, производственные цеха, городская застройка – являются динамическими: расположение препятствий и объектов в них заранее неизвестно или постоянно изменяется в ходе выполнения задачи. Это делает классические схемы планирования, требующие полной перестройки маршрута при каждом изменении карты, избыточными и неэффективными по времени отклика, что ограничивает их применимость в системах реального времени.

Задачу планирования движения в подобных условиях принято разделять на два иерархических уровня – глобальный, отвечающий за построение маршрута на известной карте, и локальный, обеспечивающий безопасный объезд внезапно возникших препятствий в режиме реального времени.

Целью данной работы является исследование методов планирования и управления в рамках задачи автономной навигации роботов в динамической среде, включающее сравнительный анализ существующих подходов и практическую реализацию выбранных алгоритмов в составе индустриального навигационного стека. На уровне глобального планирования рассматривается переход от классического поиска (A^*) к инкрементальным методам (D^* Lite), позволяющим обновлять маршрут без полного пересчёта графа состояний при локальных изменениях карты. На уровне локального управления исследуются стохастические предсказательные контроллеры MPPI и RA-MPPI, а также их модификации с явным линейным прогнозом траекторий подвижных препятствий.

Для достижения поставленной цели решены следующие задачи:

- 1 проведён обзор и классификация современных методов глобального и локального планирования; обоснован выбор D^* Lite и RA-MPPI для дальнейшего исследования;
- 2 разработана собственная программная модель сеточной карты для количественного сравнения алгоритмов по времени, длине пути, извилистости и доле опасных сближений;
- 3 проведены экспериментальные сравнения A^* и D^* Lite и MPPI/RA-MPPI;
- 4 выбранные алгоритмы реализованы в виде оригинальных плагинов навигационного стека Nav2 для ROS 2;
- 5 собранный стек проверен сквозным тестированием на симуляторе Gazebo Sim с моделью робота TurtleBot3 Burger в 16 конфигурациях.

Основными результатами работы являются: количественное сравнение глобальных планировщиков, подтвердившее существенное преимущество D^* Lite по времени отклика и его предсказуемости при интенсивных изменениях карты; сравнение локальных контроллеров, показавшее, что RA-MPPI снижает долю опасных сближений в несколько раз по сравнению с базовым MPPI ценой пропорционального увеличения вычислительных затрат, а явный прогноз подвижных препятствий даёт наибольший дополнительный выигрыш в сочетании с RA-MPPI при средней плотности динамики; два оригинальных программных пакета для ROS 2, интегрирующих D^* Lite и RA-MPPI в стек Nav2 и подтвердивших работоспособность гибридной архитектуры на симуляторе Gazebo Sim с роботом TurtleBot3 Burger.

2 Обзор литературы

В современных системах управления автономными роботами задачу навигации принято разделять на два иерархических уровня: глобальное и локальное планирование. Такое разделение обусловлено необходимостью баланса между вычислительной сложностью и скоростью реакции на изменения среды.

Глобальное планирование отвечает за построение оптимального маршрута на всей известной карте. Однако расчет такого пути требует значительных ресурсов и не может выполняться с высокой частотой. Локальное планирование, напротив, работает в ограниченном «окне» видимости сенсоров, обеспечивая безопасный объезд внезапно возникших препятствий и удержание робота на глобальной траектории в режиме реального времени. Совместное использование этих уровней позволяет роботу следовать к цели и одновременно быстро реагировать на изменения окружающей среды.

Задаче планирования движения мобильных роботов в динамических средах посвящено большое количество научных исследований. В обзорных работах [1, 2, 3], предлагаются классификации методов планирования, анализируются их достоинства и ограничения. В данной главе рассматриваются основные классы алгоритмов глобального и локального планирования с акцентом на их применимость в условиях изменяющейся среды.

2.1 Обзор методов глобального планирования

Методы глобального планирования предназначены для построения маршрута от старта до цели по известной заранее карте среды (полностью или частично). Как правило, они ориентированы на поиск оптимального или близкого к оптимальному пути и используются на верхнем уровне навигационной системы.

2.1.1 Алгоритмы поиска по графу

Алгоритмы поиска по графу относятся к классическим методам планирования и широко применяются в задачах навигации мобильных роботов [3]. Они предполагают дискретизацию пространства в виде графа или решётки и поиск пути между вершинами.

Алгоритм Дейкстры [4] находит кратчайшие пути от заданной вершины до всех остальных в графе с неотрицательными весами рёбер. Он гарантирует нахождение оптимального решения, однако при увеличении размера карты приводит к высоким накладным расходам в задачах планирования в реальном времени.

Алгоритм A^* [5] – эвристический алгоритм поиска кратчайшего пути в графе, на каждом шаге расширяющий вершину с минимальной оценкой $f(n) = g(n) + h(n)$, где $g(n)$ – пройденная стоимость пути от старта до n , а $h(n)$ – эвристическая оценка оставшейся стоимости пути до цели. При допустимой эвристике, не переоценивающей истинной стоимости до цели, A^* гарантированно находит оптимальное решение и при типичных эвристиках в задачах планирования на двумерной сетке расширяет существенно меньше вершин, чем алгоритм

Дейкстры. В высокоразмерных пространствах эффективность A^* падает из-за сложности построения информативной эвристики.

В условиях динамической среды классические алгоритмы Дейкстры и A^* требуют полного пересчёта маршрута при изменении информации о препятствиях, что ограничивает их применение в реальном времени.

Для устранения данного недостатка были разработаны инкрементальные алгоритмы перепланирования. **Алгоритм D^*** [6] обновляет ранее найденный путь при изменении карты, повторно используя результаты предыдущего поиска вместо полного пересчёта. **Lifelong Planning A^* (LPA*)** [7] обобщает эту идею для повторных поисков с фиксированными стартом и целью при меняющихся стоимостях рёбер. **D^* Lite** [8] – переформулировка D^* через LPA* с обратным направлением поиска от цели, существенно более простая для реализации и анализа. **ARA* (Anytime Repairing A^*)** [9] реализует anytime-подход: быстро получает ε -приближённое решение и итеративно улучшает его при последующих вызовах, уменьшая ε .

2.1.2 Алгоритмы случайной выборки (Sampling-based)

Алгоритмы случайной выборки (sampling-based planners) предназначены для планирования движения в непрерывных и высокоразмерных пространствах конфигураций [10]. Они основаны на случайном семплировании допустимых состояний и построении графа или дерева достижимых конфигураций.

Алгоритм Rapidly-exploring Random Tree (RRT) [11] итеративно расширяет дерево достижимых конфигураций: на каждом шаге выбирается случайная точка пространства, в дереве находится ближайшая к ней вершина и от неё в её направлении добавляется новое ребро ограниченной длины. За счёт такого выбора дерево естественно растёт в сторону неисследованных областей, что обеспечивает быстрое покрытие пространства. Однако получаемые пути, как правило, далеки от оптимальных.

Алгоритм RRT* [12] – модификация RRT, добавляющая шаг переподключения: после добавления новой вершины в дереве пересматриваются связи близлежащих вершин и при необходимости их родитель меняется на новую вершину, если это укорачивает путь от старта. За счёт этого механизма RRT* асимптотически оптимален: с ростом числа сэмплов качество пути сходится к оптимальному, тогда как базовый RRT такой гарантии не даёт. На практике эта сходимость может быть медленной, особенно в средах с узкими проходами.

Алгоритм Batch Informed Trees (BIT*) [13] объединяет идеи RRT* и A^* : пространство сэмплируется не по одной точке, как в RRT*, а батчами по N точек, образующих неявный граф, по которому ведётся эвристический поиск в стиле A^* (вершины раскрываются в порядке $f(n) = g(n) + h(n)$). После каждой итерации текущая стоимость лучшего пути используется для ограничения области последующего сэмплирования, что фокусирует поиск на потенциально улучшающих решениях. BIT* асимптотически оптимален и на практике сходится к оптимальному пути быстрее RRT*. **Алгоритм RABIT*** [14] – расширение BIT*, встраивающее в глобальный сэмплинг-поиск шаг локальной оптимизации траектории:

предложенные сэмплированием рёбра могут быть локально доработаны перед включением в дерево. Это позволяет дорогостоящему локальному оптимизатору работать только над перспективными рёбрами и ускоряет сходимость к оптимальному решению, особенно в средах со сложной структурой препятствий.

Для задач с динамическими препятствиями предлагаются специализированные методы, например **Risk-DTRRT** [15], учитывающий вероятность столкновений и уровень риска при построении траектории.

2.1.3 Интеллектуальные бионические алгоритмы

Бионические и интеллектуальные алгоритмы основаны на моделировании коллективного поведения биологических систем и применяются для решения задач оптимизации [16]. В планировании движения они используются для поиска приближённых оптимальных траекторий.

Генетические алгоритмы (GA) [17] – эволюционная схема поиска, в которой кандидатные решения кодируются в виде «хромосом» и подвергаются операциям отбора, скрещивания и мутации. В задачах планирования траектория обычно представляется последовательностью промежуточных точек, а функция приспособленности учитывает длину пути, гладкость и наличие столкновений с препятствиями. ГА гибко настраиваются на целевую функцию и кодирование, однако требуют значительных вычислительных ресурсов и не гарантируют сходимости за ограниченное время.

Алгоритмы муравьиной колонии (ACO) [18] – мета-эвристика, в которой искусственные муравьи итеративно строят решения, выбирая каждый шаг с вероятностью, пропорциональной уровню виртуального феромона на ребре. После каждой итерации феромон испаряется, а на рёбрах лучших решений усиливается, что создаёт положительную обратную связь и концентрирует поиск вокруг качественных маршрутов. ACO эффективен для поиска кратчайших маршрутов (изначально предложен для задачи коммивояжёра), однако чувствителен к настройке параметров и медленно адаптируется к динамическим изменениям среды.

Алгоритм искусственной пчелиной колонии (ABC) [19] моделирует поведение пчёл-фуражиров: занятые пчёлы исследуют известные источники нектара (кандидатные решения), наблюдатели выбирают перспективные источники с вероятностью, пропорциональной их качеству, а разведчики случайным образом инициализируют новые источники, когда старые перестают улучшаться. **Алгоритм роя частиц (PSO)** [20] использует подвижный рой кандидатных решений: каждая «частица» обновляет свою скорость в направлении собственной лучшей позиции и лучшей позиции в рое, постепенно концентрируя поиск вокруг наиболее перспективной области. Оба метода применяются в планировании для оптимизации параметрических представлений траектории, легко параллелятся, но существенно зависят от начальной инициализации и параметров.

2.2 Обзор методов локального планирования

Методы локального планирования предназначены для формирования управляющих воздействий в реальном времени на основе текущего состояния робота и информации о ближайшем окружении. В отличие от глобальных планировщиков, которые строят маршрут на всей карте, локальные методы работают в ограниченной области и ориентированы прежде всего на безопасное движение, обход препятствий и следование заданному глобальному пути. Это делает их особенно важными в динамических средах, где присутствуют движущиеся объекты и неопределённость в данных сенсоров.

2.2.1 Классические контроллеры

Классические контроллеры применяются в первую очередь для задачи слежения за заданной траекторией и стабилизации движения робота. Они не выполняют планирование траектории в строгом смысле, однако широко используются как нижний исполнительный уровень в навигационных системах.

PID-контроллер является одним из наиболее простых и распространённых регуляторов. Он формирует управляющее воздействие на основе текущей ошибки, её интегральной и дифференциальной составляющих. В задачах мобильной робототехники PID применяется для управления линейной и угловой скоростью робота с целью достижения заданной позиции или следования траектории [21]. Основными преимуществами PID-регулятора являются простота реализации, низкие вычислительные затраты и высокая частота работы. К недостаткам относится отсутствие учёта динамики окружающей среды и невозможность самостоятельного избегания препятствий, что ограничивает его применение только исполнительным уровнем.

Линейно-квадратичный регулятор (LQR) основан на минимизации квадратичного функционала качества, учитывающего отклонение состояния системы от заданного и величину управляющего воздействия. В работах [22] и [23] LQR применяется для управления мобильным роботом, в том числе для обхода препятствий и слежения за траекторией. LQR учитывает модель динамики системы, что повышает точность управления. Его основным недостатком является необходимость линеаризации модели в случае нелинейной системы и чувствительность к ошибкам параметров.

Комбинированные схемы PID+LQR [24] используются для объединения простоты PID и оптимальных свойств LQR: PID обеспечивает базовое слежение за заданием, а LQR – стабилизирующее воздействие на основе модели системы. Гибридные контроллеры требуют более сложной настройки и точного согласования параметров.

В целом классические контроллеры не являются полноценными локальными планировщиками, но выполняют важную роль исполнительного уровня, обеспечивая реализацию траекторий, полученных более высокими уровнями навигационной системы.

2.2.2 Геометрические или реактивные методы

Геометрические или реактивные методы локального планирования принимают решения непосредственно на основе текущих сенсорных данных и геометрии окружающей среды. Они не строят долгосрочные траектории, а выбирают допустимое направление или скорость движения, обеспечивающие безопасность в ближайшем будущем.

Метод Dynamic Window Approach (DWA) был предложен в [25]. Он рассматривает пространство допустимых линейных и угловых скоростей робота и выбирает те из них, которые достижимы с учётом динамических ограничений и не приводят к столкновениям. Далее используется целевая функция, учитывающая приближение к цели, скорость движения и расстояние до препятствий. В работе [26] предложена модификация DWA, ориентированная на динамические препятствия и групповые движения роботов. Основным достоинством DWA является высокая скорость работы и практическая применимость.

Метод Velocity Obstacles (VO) был предложен в [27]: в пространстве скоростей робота строится множество тех управлений, которые при текущих скоростях препятствий приведут к столкновению, и среди оставшихся выбирается ближайшая к желаемой. В работе [28] VO встроен в гибридный планировщик, где движение препятствий предсказывается по облаку точек и подаётся на вход VO. Метод корректно работает в динамических средах, однако опирается на предположение о постоянстве скоростей препятствий.

Метод искусственных потенциальных полей (APF) был предложен Хатибом в [29]. Цель моделируется как источник притяжения, а препятствия — как источники отталкивания, и робот движется по направлению результирующего градиента. В [30] предложена модификация APF для задач навигации в неизвестной среде с динамическими препятствиями. Основными преимуществами APF являются простота и высокая вычислительная эффективность.

Метод Vector Field Histogram (VFH) был представлен в [31] и основан на построении гистограммы плотности препятствий по данным сенсоров. На её основе выбирается направление движения с минимальной опасностью столкновения. В работе [32] предложены модификации VFH, улучшающие точность построения гистограмм и учитывающие динамические препятствия. Метод хорошо работает в реальном времени, однако не обеспечивает глобальной оптимальности траектории.

Геометрические методы отличаются высокой вычислительной эффективностью и широко применяются в реальных робототехнических системах. Их основным недостатком является локальный характер принятия решений, что может приводить к неоптимальному поведению в сложных средах.

2.2.3 Оптимизационные или предсказательные методы

Оптимизационные методы формулируют задачу локального планирования как задачу оптимального управления, в которой необходимо минимизировать функционал качества при наличии динамических и кинематических ограничений.

Model Predictive Control (MPC) является одним из наиболее распространённых методов этого класса. В обзоре [33] показано, что MPC позволяет учитывать динамику робота, ограничения на управление и наличие препятствий. Алгоритм на каждом шаге решает задачу оптимизации на конечном горизонте прогнозирования, обеспечивая высокое качество траекторий. Основным недостатком является высокая вычислительная сложность.

Model Predictive Path Integral (MPPI) [34] – стохастический вариант MPC, в котором на каждом шаге выбирается большое число возмущённых траекторий, каждая оценивается по функции стоимости, после чего управление формируется как взвешенная комбинация сэмплов с весами Больцмана – экспоненциально убывающими по стоимости траектории, так что более качественные траектории вносят больший вклад в итоговое управление. В работе [35] MPPI успешно применён для задачи агрессивного автономного вождения. При интеграции в навигационный стек MPPI часто опирается на локальную карту стоимостей (costmap), формируемую из заранее известной карты и данных сенсоров: в функцию стоимости каждой сэмплированной траектории напрямую включается штраф за столкновение с занятыми клетками costmap [36], что позволяет интегрировать глобальную и локальную информацию о среде. Преимуществами MPPI являются возможность работы с нелинейной динамикой и со сложными функциями стоимости. Недостатком является необходимость больших вычислительных ресурсов.

Алгоритм CCS-MPPI, предложенный в [37], объединяет MPPI с методом Covariance Steering и позволяет управлять не только средним значением состояния, но и его ковариацией, обеспечивая вероятностные гарантии соблюдения ограничений. Этот подход особенно эффективен в стохастических средах, но сложен в реализации.

Метод RMPPI (Robust MPPI), предложенный в [38], направлен на повышение робастности MPPI к ошибкам модели и внешним возмущениям. Он сочетает MPPI с трекинг-контроллером и механизмами безопасного распространения номинальной траектории, для которых доказаны аналитические гарантии ограниченности ошибки отслеживания при ограниченных возмущениях.

Метод RA-MPPI (Risk-Aware MPPI), предложенный в [39], повышает робастность MPPI – устойчивость качества управления к отклонениям реальной системы от используемой модели и к внешним возмущениям – за счёт явного учёта хвостового риска (ожидаемых потерь в редких, но катастрофических исходах) на основе меры CVaR (Conditional Value-at-Risk), равной среднему значению стоимости среди заданного процента худших возмущённых сценариев. Для каждой номинальной траектории разыгрывается серия возмущённых роллаутов – симуляций траектории на горизонте прогноза, в которых к управлению добавляется случайный шум, отражающий неопределённость его исполнения. CVaR по этим роллаутам добавляется как штраф к стоимости траектории, смещая взвешивание Больцмана (формирование управления как взвешенной комбинации сэмплов с экспоненциально убывающими по стоимости весами) в сторону траекторий с низким хвостовым риском без полного исключения рискованных сэмплов из рассмотрения.

Метод Adaptive MPPI, предложенный в работе [40], автоматически подбирает па-

параметры модели и контроллера с помощью байесовской оптимизации – метода глобальной оптимизации, в котором строится вероятностная модель целевой функции и итеративно выбираются наиболее информативные точки её вычисления; в данной работе она аппроксимируется обучением классификатора. Это повышает устойчивость алгоритма при неточной модели динамики и гетероскедастическом шуме.

Оптимизационные методы обеспечивают наивысшее качество траекторий и наилучший учёт динамики робота и среды, однако их применение ограничено высокими вычислительными затратами и сложностью реализации.

2.3 Сравнительный анализ и выбор алгоритмов

Для систематизации полученных данных и выбора оптимального стека технологий был проведен сравнительный анализ алгоритмов. В таблицах [2.1](#), [2.2](#) представлены ключевые характеристики как глобальных, так и локальных планировщиков.

Для оценки эффективности используются следующие обозначения:

«+» — высокий показатель (высокая скорость работы, эффективная работа с динамикой, высокая точность);

«+-» — средний показатель (зависит от настроек, средние вычислительные затраты);

«-» — низкий показатель (медленная работа, неэффективность в динамике, риск локальных минимумов).

Таблица 2.1: Сравнение алгоритмов глобального планирования

Класс/ алгоритм	Работа в динамике	Скорость работы	Основные преимущества	Основные недостатки
Графовые Dijkstra	—	—	Гарантированная оптимальность, простота	Полный пересчёт при изменениях
Графовые A*	—	+	Быстрее Дейкстры, эвристический поиск	Расход памяти на крупных картах; не использует предыдущие поиски при пересчёте
Графовые D*	+	+-	Первый инкрементальный планировщик для динамической среды	Сложная реализация
Графовые D* Lite	+	+-	Проще в реализации и анализе, чем D*	Сложнее A* при разовом построении
Графовые LPA*	+-	+-	Инкрементальный пересчёт A* при изменении весов рёбер	Применим только при изменении весов рёбер, не при структурных изменениях карты
Графовые ARA*	—	+-	Быстрое первое решение	Не обрабатывает изменения карты во время выполнения
Семплинг RRT	—	+	Быстро находит путь	Плохое качество пути
Семплинг RRT*	—	—	Оптимальные траектории	Медленная сходимость
Семплинг BIT*	—	+-	Эвристическое ускорение, быстрее RRT*	Сложная настройка
Семплинг RABIT*	—	+-	Быстрая оптимизация	Зависит от локального оптимизатора
Семплинг Risk-DTRRT	+	—	Учёт неопределённости	Сложность модели, высокие вычислительные затраты
Бионические GA	—	—	Гибкость, адаптация	Медленная сходимость
Бионические ACO	—	+-	Распределённый поиск	Медленная адаптация к динамике
Бионические ABC	+-	+-	Простота реализации, баланс разведки и эксплуатации	Чувствительность к параметру limit (порог отказа от источника)
Бионические PSO	+-	+-	Быстро сходится	Преждевременная сходимость к локальному оптимуму

Таблица 2.2: Сравнение алгоритмов локального планирования

Класс/ алгоритм	Работа в динамике	Скорость работы	Основные преимущества	Основные недостатки
Классические контроллеры PID	—	+	Простота, высокая частота управления	Нет планирования, нет обхода препятствий
Классические контроллеры LQR	+—	+—	Оптимальность, устойчивость	Применим только к линейным моделям системы
Классические контроллеры PID+LQR	+—	+—	Повышенная устойчивость и точность	Сложность настройки
Геометрические DWA	+	+	Поиск в пространстве скоростей с учётом динамических ограничений	Без глобального обзора — ограниченный горизонт прогноза
Геометрические VO	+	+—	Учёт движущихся препятствий	Допущение постоянства скоростей препятствий
Геометрические APF	+—	+	Простота, высокая скорость	Колебания у препятствий
Геометрические VFH	+	+	Эффективная работа с сенсорами	Нет глобальной оптимальности
Оптимизационные MPC	+	—	Учёт ограничений и динамики	Большие вычислительные затраты
Оптимизационные MPPI	+	—	Работает с произвольной нелинейной динамикой и стоимостной функцией	Большой вычислительный бюджет на каждом шаге
Оптимизационные CCS-MPPI	+	—	Вероятностные гарантии безопасности	Сложная реализация
Оптимизационные RMPPI	+	—	Доказанные гарантии ограниченности ошибки	Требует трекинг-контроллера
Оптимизационные RA-MPPI	+	—	Снижает долю опасных сближений с препятствиями	Кратное увеличение вычислительных затрат относительно MPPI
Оптимизационные Adaptive MPPI	+	—	Адаптация к неопределённости	Сложность настройки

2.3.1 Выбор алгоритмов

Для решения задачи навигации в динамических средах в качестве глобального планировщика выбран алгоритм D* Lite, в качестве локального — RA-MPPI.

В условиях непрерывного обновления карты сенсорными данными использование классического A* становится неэффективным: необходимость полной перестройки графа при каждом изменении среды создает критические вычислительные задержки. Алгоритм D* Lite нивелирует этот недостаток за счет механизма инкрементального поиска, при котором пересчитываются только участки пути, непосредственно затронутые изменениями. Это позволяет достичь оптимального баланса между глобальной эффективностью маршрута и высокой скоростью реакции, характерной для локальных планировщиков. Кроме того, относительная алгоритмическая простота D* Lite упрощает его интеграцию в гибридные навигационные системы и облегчает процесс отладочного тестирования.

На уровне локального управления базовый MPPI обеспечивает работу с произвольной нелинейной динамикой робота и со сложной функцией стоимости, однако не имеет явного механизма учёта риска: рискованные траектории взвешиваются наряду с безопасными, что в

условиях плотной динамики приводит к повышенной доле опасных сближений с препятствиями. Алгоритм RA-MPPI устраняет этот недостаток, добавляя к стоимости каждой кандидатной траектории дополнительный штраф за её поведение в худших случаях. Для оценки этого штрафа алгоритм многократно симулирует каждую траекторию со случайными возмущениями управления и усредняет стоимость по наиболее неблагоприятным реализациям; в результате траектории, которые могут привести к опасным ситуациям при отклонении исполнения от плана, получают увеличенный штраф и реже выбираются итоговым управлением. Это позволяет интерпретируемо контролировать баланс между скоростью движения и безопасностью одним параметром, не вводя жёстких ограничений на пространство управлений. По сравнению с альтернативными модификациями MPPI (RMPPI, CCS-MPPI) RA-MPPI обладает существенно более простой реализацией: он не требует дополнительного контроллера для отслеживания номинальной траектории или решения отдельных задач оптимизации на каждом шаге, что упрощает его интеграцию в плагин Nav2 и облегчает отладку.

2.4 Выводы по главе

Проведённый анализ подтверждает отсутствие универсального алгоритма, способного одновременно обеспечивать оптимальный глобальный маршрут и эффективное реагирование на изменения среды. Графовые планировщики гарантируют достижение целевой точки, однако классические их представители (A^* , Dijkstra) не справляются с динамической средой: при каждом изменении карты требуется полная перестройка маршрута. Сэмплирующие методы (RRT, RRT*, BIT*, RABIT*) и бионические алгоритмы (GA, ACO) также не предназначены для оперативного перепланирования. Единственным классом, сочетающим инкрементальное перепланирование с приемлемой скоростью отклика, оказались инкрементальные графовые планировщики семейства D^* (D^* , D^* Lite, частично LPA*).

Локальные планировщики, напротив, ориентированы на реактивное управление в режиме реального времени. Простые геометрические методы (DWA, VFH, APF) обеспечивают высокую скорость, но действуют только в пределах локального окна и не учитывают неопределённость исполнения управления. Оптимизационные методы (MPC, MPPI) учитывают динамику робота и сложную функцию стоимости ценой значительных вычислительных затрат; их риск-чувствительные модификации (RMPPI, CCS-MPPI, RA-MPPI) дополнительно явно учитывают надёжность исполнения управления, что особенно важно в плотной динамической среде.

Наиболее эффективным решением для рассматриваемой задачи признана гибридная архитектура: за инкрементальный пересчёт маршрута отвечает D^* Lite, а за безопасную отработку траектории – риск-чувствительный локальный контроллер RA-MPPI. Такое сочетание минимизирует простой робота при изменениях карты, явно учитывает неопределённость исполнения управления и обеспечивает баланс между глобальной оптимальностью маршрута и оперативной реакцией на динамические препятствия.

3 Алгоритмы глобального планирования

В данной главе рассматриваются алгоритмы глобального планирования, выбранные в предыдущей главе для подробного исследования: классический A^* и инкрементальный D^* Lite. Сначала описаны основные концепции их работы и принципиальные различия в поведении при изменениях карты, после чего приводится экспериментальное сравнение в статической и динамической средах.

3.1 Общие концепции алгоритмов A^* и D^* Lite

Оба рассматриваемых алгоритма решают задачу поиска кратчайшего пути в графе. Состоянием системы является позиция робота на клеточной карте (или вершина графа состояний), действиями – переходы к соседним клеткам с известной стоимостью. Принципиальное различие алгоритмов состоит в стратегии работы с информацией о предыдущих поисках при изменениях карты.

Алгоритм A^* . Классический A^* выполняет поиск в прямом направлении: от стартовой вершины s_{start} к цели s_{goal} . На каждом шаге алгоритм поддерживает два множества: *открытое* (Open) – вершин, известных, но ещё не раскрытых, и *закрытое* (Closed) – вершин, уже раскрытых. Для каждой вершины n хранится текущая оценка $g(n)$ – стоимость пути от старта до n через известных предшественников – и вычисляется приоритет $f(n) = g(n) + h(n)$, где $h(n)$ – эвристическая нижняя оценка расстояния от n до цели.

Пошаговая схема работы алгоритма:

- 1 Положить $g(s_{\text{start}}) = 0$, добавить s_{start} в Open с приоритетом $f(s_{\text{start}}) = h(s_{\text{start}})$.
- 2 Пока Open не пуст:
 - 2.1. Извлечь из Open вершину u с минимальным $f(u)$.
 - 2.2. Если $u = s_{\text{goal}}$ – путь найден, восстановить его обратным проходом по сохранённым родителям.
 - 2.3. Иначе перенести u в Closed.
 - 2.4. Для каждого соседа v вершины u вычислить кандидатную оценку $g_{\text{new}} = g(u) + c(u, v)$, где $c(u, v)$ – стоимость перехода. Если $g_{\text{new}} < g(v)$, обновить $g(v) = g_{\text{new}}$, установить u родителем v и добавить v в Open с приоритетом $f(v) = g(v) + h(v)$.

При допустимой эвристике (не переоценивающей истинной стоимости) A^* гарантированно находит оптимальный путь. В статической среде это один из самых эффективных алгоритмов поиска. Однако при изменении карты A^* не имеет механизма обновления накопленных результатов: единственный способ адаптации – полный перезапуск с нуля.

Алгоритм D* Lite. В отличие от A^* , D^* Lite ведёт поиск в обратном направлении – от цели s_{goal} к старту s_{start} , что естественно подходит мобильному роботу: цель остаётся неподвижной, а робот (точка старта) движется. Для каждой вершины n алгоритм хранит две величины:

- $g(n)$ – текущая оценка стоимости пути от n до цели;
- $rhs(n)$ – *одношаговая* оценка, полученная как $\min_{n' \in \text{succ}(n)} (c(n, n') + g(n'))$.

Вершина n называется *локально согласованной*, если $g(n) = rhs(n)$, и *локально несогласованной* иначе. Алгоритм поддерживает очередь приоритетов, содержащую только локально несогласованные вершины – именно те, в которых накоплены нераспространённые изменения.

Пошаговая схема работы алгоритма:

- 1 Инициализировать $g(n) = rhs(n) = \infty$ для всех вершин, $rhs(s_{\text{goal}}) = 0$. Добавить s_{goal} в очередь.
- 2 Извлекать из очереди вершины с минимальным приоритетом и обрабатывать их:
 - 2.1. Если $g(u) > rhs(u)$ (переоценено) – установить $g(u) = rhs(u)$ и обновить всех предшественников u .
 - 2.2. Если $g(u) < rhs(u)$ (недооценено) – установить $g(u) = \infty$ и обновить u и его предшественников.
- 3 При изменении стоимости ребра (u, v) – обновить $rhs(u)$, перепроверить локальную согласованность u и при необходимости поместить u в очередь.
- 4 Текущий маршрут робота: на каждом шаге переходить из текущего положения s_{start} в соседнюю вершину s' , минимизирующую $c(s_{\text{start}}, s') + g(s')$.

Ключевое свойство D^* Lite: при локальном изменении карты повторное вычисление затрагивает только те вершины, чьи g - или rhs -значения оказались несогласованными. Если изменения локализованы вблизи маршрута, объём пересчёта оказывается малым по сравнению с полным запуском поиска. Это и обеспечивает преимущество D^* Lite над A^* в динамических средах: A^* при каждом изменении выполняет независимый полный поиск, тогда как D^* Lite извлекает выгоду из инвариантной части предыдущих вычислений.

3.2 Модель и условия эксперимента

Целью исследования было сравнение эффективности классического и инкрементального подходов в различных условиях навигации.

В качестве карты использовалась модифицированная версия бенчмарка «Arena» с ресурса MovingAI [41] – открытое пространство размером 49×49 клеток с блоками статических препятствий и распределёнными группами зон спавна динамических объектов (рис. 3.1).

Чёрные клетки соответствуют непроходимым статическим препятствиям, белые – свободно-
му пространству, красные точки – стартовым позициям подвижных препятствий.

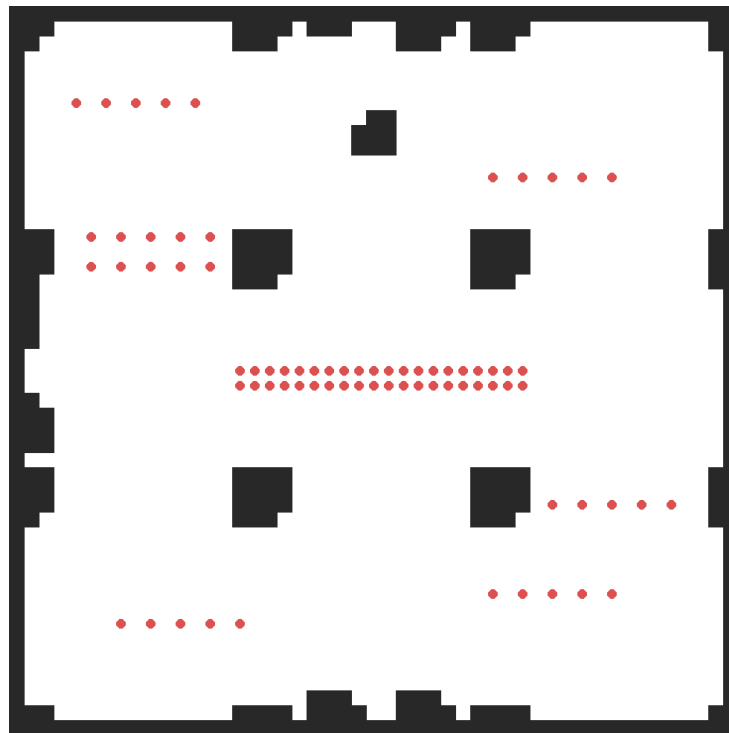


Рис. 3.1: Модифицированная карта «Arena» размером 49×49 клеток, использованная при сравнении A^* и D^* Lite.

Параметр сложности конфигурации (n_dst) определялся как минимальное манхэттенское расстояние между точками старта и финиша: $d_M = |x_{start} - x_{goal}| + |y_{start} - y_{goal}| \geq n$. Для каждой итерации (от 0_dst до 70_dst) генерировалось 1000 случайных пар точек, что обеспечило высокую статистическую достоверность.

В ходе тестов от 1 до 15 препятствий за каждый ход совершали случайное перемещение на одну клетку (вверх, вниз, влево или вправо). Для алгоритма A^* проверялись две стратегии пересчёта: в режиме *step* путь пересчитывается один раз в конце хода после того, как все препятствия совершили движение; в режиме *change* путь пересчитывается мгновенно после каждого единичного перемещения любого препятствия.

3.3 Результаты экспериментов

Объединённые результаты экспериментов представлены на рисунке 3.2. По оси абсцисс отложено число подвижных препятствий за ход (значение 0 соответствует статической среде, 1–15 – различным уровням динамики); по оси ординат – среднее время построения или обновления маршрута в миллисекундах. Значения усреднены по всем пяти конфигурациям минимальной дистанции между стартом и целью (0_dst , 10_dst , 30_dst , 50_dst , 70_dst). Для алгоритма A^* приведены два режима перепланирования: *step* (одно перепланирование в конце хода) и *change* (перепланирование после каждого единичного перемещения препятствия). Алгоритм D^* Lite показан одной кривой, поскольку в обоих режимах его поведение

почти не различается.

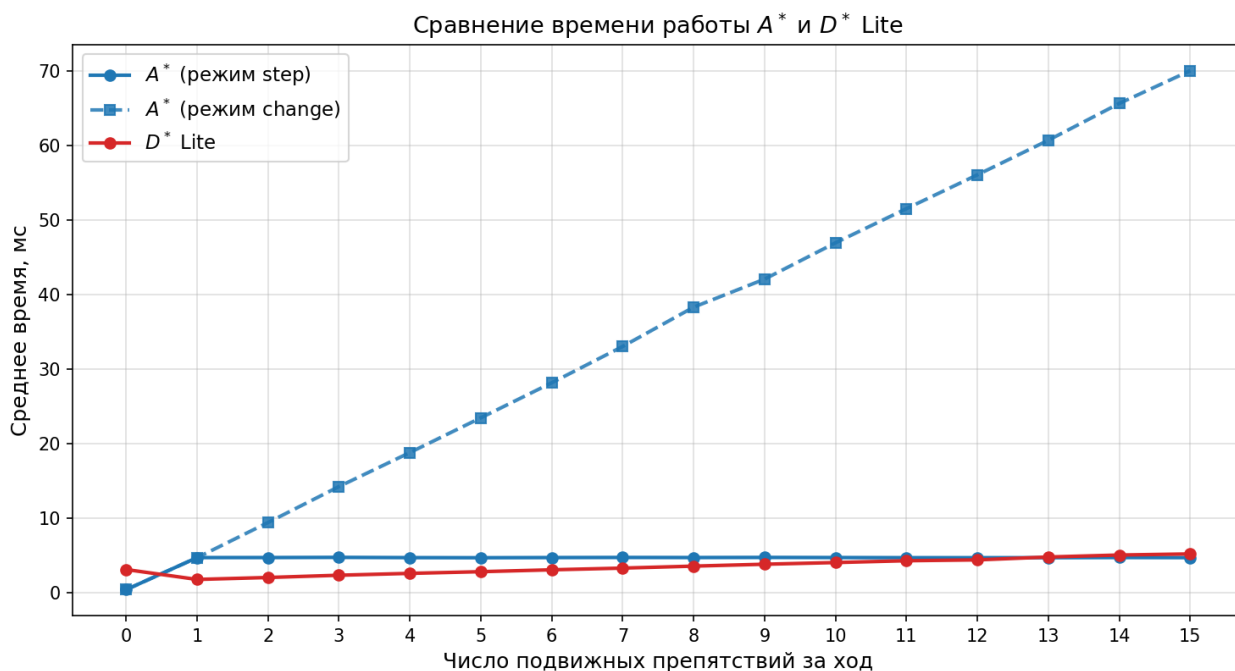


Рис. 3.2: Сравнение времени работы A^* и D^* Lite при различной интенсивности изменений карты (усреднение по всем дистанциям между стартом и целью).

3.4 Анализ производительности в статической среде

В условиях неизменной карты (значение 0 на оси абсцисс рис. 3.2) алгоритм A^* показал преимущество. Его вычислительная сложность при разовом поиске значительно ниже, чем у конкурента: среднее время поиска по всем дистанциям составляет около 0,5 мс, что подтверждает оптимальность A^* для однократного планирования на статичной карте.

D^* Lite демонстрирует более тяжеловесный старт – около 3 мс в статике. Это связано с накладными расходами на инициализацию сложных структур данных и приоритетных очередей, которые необходимы алгоритму для будущей адаптации к изменениям, но в статике являются избыточными.

3.5 Анализ производительности в динамической среде

При введении движущихся препятствий ситуация кардинально меняется (правая часть рис. 3.2). Ключевым фактором становится способность алгоритма переиспользовать предыдущие вычисления.

Алгоритм A^* не обладает «памятью» о предыдущих итерациях, что приводит к полной перепланировке при каждом изменении карты. В режиме *change* среднее время пересчёта растёт линейно с числом изменений за ход: от ≈ 9 мс при 1 препятствии до ≈ 70 мс при 15 препятствиях, поскольку алгоритм выполняет N полноценных поисков за один цикл. Режим *step* снижает нагрузку до $\approx 4,5$ мс независимо от числа препятствий (одно перепланирование на ход) и оказывается сопоставимым с D^* Lite по времени при высокой плотности изменений.

Однако применимость режима *step* в реальных условиях ограничена: между моментами пересчёта робот следует устаревшему маршруту, полученному до перемещения препятствий, и не реагирует на промежуточные изменения карты. При высокой плотности динамики это означает, что часть хода робот движется по маршруту, который уже мог стать невалидным (например, проходит через клетку, занятую сместившимся препятствием), что повышает риск столкновения и приводит к небезопасным траекториям. В реализованной модели данная проблема исключена за счёт фиксированного порядка ходов: сначала перемещаются препятствия, затем робот пересчитывает маршрут и делает шаг – всё в пределах одного такта моделирования. Режим *change*, напротив, гарантирует реакцию на каждое изменение, но при A^* обходится прямо пропорциональным ростом вычислительной стоимости. D^* Lite по существу обеспечивает поведение режима *change* (реакцию на каждое изменение) ценой режима *step* (постоянное время на ход), что и составляет его практическое преимущество.

Алгоритм D^* Lite специально разработан для работы в меняющемся окружении. Его среднее время обработки изменений растёт очень умеренно – от $\approx 1,7$ мс при 1 препятствии до ≈ 5 мс при 15 препятствиях. На карте «Arena» это проявляется наиболее ярко: поскольку объекты перемещаются всего на одну клетку, D^* Lite обновляет лишь локальные веса в своём графе, не затрагивая те части пути, которые остались актуальными. Это делает его в 12–15 раз быстрее A^* в режиме *change* при высокой плотности динамики.

Наблюдаемые результаты подтверждают принципиальное различие вычислительных стратегий алгоритмов. В статической среде A^* демонстрирует явное превосходство благодаря элегантной архитектуре поиска в графе. Однако в динамической среде при росте числа препятствий и частоте обновлений преимущество полностью переходит к D^* Lite за счет инкрементального характера обновлений. Разрыв в производительности становится катастрофичным для A^* в режиме *change*: при 15 движущихся объектах производительность падает по крайней мере в 15...20 раз по сравнению с D^* Lite.

3.6 Выводы по главе

Проведенное сравнение показало, что выбор алгоритма существенно зависит от характера среды и требований приложения.

Таблица 3.1: Сравнительные характеристики A^* и D^* Lite

Параметр сравнения	A^* (A-Star)	D^* Lite
Статическая скорость (мин–макс, среднее)	0,22–0,74 мс ($\sim 0,46$ мс)	1,26–5,69 мс ($\sim 3,16$ мс)
Динамика: режим <i>step</i> (мин–макс, среднее)	1,98–10,70 мс ($\sim 4,76$ мс)	0,92–9,29 мс ($\sim 3,59$ мс)
Динамика: режим <i>change</i> (мин–макс, среднее)	2,04–155,58 мс ($\sim 37,55$ мс)	0,92–9,33 мс ($\sim 3,59$ мс)

Для разового построения пути в статических условиях классический A^* эффективен благодаря простоте и низким накладным расходам. Однако для задач с регулярными изменениями карты, особенно при интенсивной динамике, значительно более устойчивым по времени выполнения оказывается D^* Lite.

Таким образом, эксперимент подтверждает, что для задач разовой навигации предпочтителен A^* благодаря его вычислительной легкости. Однако в условиях интенсивной динамики и необходимости мгновенной реакции на изменения среды (режим *change*) D^* Lite является единственным масштабируемым решением, гарантирующим предсказуемое время отклика.

4 Алгоритмы локального планирования

В данной главе рассматриваются алгоритмы локального планирования, выбранные в обзоре для подробного исследования: базовый стохастический предсказательный контроллер MPPI и его риск-чувствительное расширение RA-MPPI, а также их модификации с явным прогнозом подвижных препятствий (далее MPPI-pred и RA-MPPI-pred). Сначала описаны формальные модели обоих алгоритмов и их пошаговые схемы, затем приводится экспериментальное сравнение в статической и динамической средах.

4.1 Общие концепции алгоритмов MPPI и RA-MPPI

Оба рассматриваемых алгоритма относятся к семейству стохастических предсказательных контроллеров. На каждом шаге оптимизация ведётся не аналитически, а по выборке возмущённых траекторий; решение получается как взвешенная комбинация сэмплов, где более качественные траектории получают больший вес.

Алгоритм MPPI. MPPI (Model Predictive Path Integral) — стохастический локальный планировщик, основанный на одновременной оценке большого числа возможных траекторий [34, 35]. На каждом шаге алгоритм оценивает K *возмущённых траекторий* — пробных траекторий, разыгрываемых вперёд по модели динамики робота из его текущей позиции: на каждом шаге горизонта $t = 1, \dots, T$ к *номинальному управлению* $\bar{u}_t$ — базовой последовательности управляющих воздействий, относительно которой ведётся сэмплирование (в данной реализации это единичный вектор, направленный к цели; в общем случае MPPI использует управляющую последовательность, оптимизированную на предыдущем такте) — добавляется гауссовский шум $\varepsilon_t^{(k)} \sim \mathcal{N}(0, \sigma^2 I)$, после чего получившееся управление $\bar{u}_t + \varepsilon_t^{(k)}$ подаётся в модель динамики и определяет следующее состояние робота. Каждая такая траектория оценивается скалярной функцией стоимости

$$J_k = \sum_{t=1}^T \left[c_{\text{obst}} \cdot \mathbb{I} \left[s_t^{(k)} \in \mathcal{O} \right] + w_s \cdot d(s_t^{(k)}, g) \right] + w_T \cdot d(s_T^{(k)}, g),$$

где $s_t^{(k)}$ — состояние k -й траектории на шаге t , $\mathcal{O}$ — множество клеток-препятствий, $\mathbb{I}[\cdot]$ — индикатор (равен 1, если условие выполнено, и 0 иначе), $d(s, g)$ — евклидово расстояние от состояния s до цели g , w_s и w_T — весовые коэффициенты промежуточного и терминального слагаемых, c_{obst} — штраф за столкновение с препятствием. Терминальное слагаемое $w_T \cdot d(s_T^{(k)}, g)$ суммируется поверх последнего слагаемого ряда и тем самым усиливает значимость финальной точки траектории.

После того как все траектории оценены, управление обновляется как взвешенная сумма возмущений, где каждая траектория получает *вес Больцмана* — экспоненциально убыва-

ющий по стоимости и нормированный так, чтобы веса в сумме давали единицу:

$$w_k = \frac{\exp\left(-\frac{J_k - J_{\min}}{\lambda}\right)}{\sum_{j=1}^K \exp\left(-\frac{J_j - J_{\min}}{\lambda}\right)}, \quad \sum_{k=1}^K w_k = 1.$$

Чем меньше стоимость траектории, тем больший вклад она вносит в итоговое управление. Параметр $\lambda > 0$ задаёт «температуру» распределения: при малых λ управление определяется преимущественно одной траекторией с минимальной стоимостью (эквивалент жёсткого $\arg \min_k J_k$), при больших – равномерно усредняется по всему ансамблю. Вычитание $J_{\min} = \min_j J_j$ из аргумента экспоненты не меняет значений нормированных весов, но предотвращает переполнение при больших стоимостях.

Пошаговая схема работы алгоритма на каждом такте управления:

- 1 Зафиксировать номинальное управление $\bar{u}$ (направление к цели или ранее выбранное управление).
- 2 Сгенерировать K возмущённых траекторий: для каждой $k = 1, \dots, K$
 - 2.1. на каждом шаге горизонта $t = 1, \dots, T$ взять гауссовское возмущение $\varepsilon_t^{(k)} \sim \mathcal{N}(0, \sigma^2 I)$;
 - 2.2. развернуть траекторию $s_0^{(k)}, s_1^{(k)}, \dots, s_T^{(k)}$, применяя управление $\bar{u}_t + \varepsilon_t^{(k)}$ через модель динамики;
 - 2.3. вычислить стоимость J_k по формуле выше.
- 3 Найти $J_{\min} = \min_j J_j$ и вычислить веса Больцмана w_k по формуле выше.
- 4 Сформировать обновлённое управление как взвешенную сумму возмущений: $u_t^* = \bar{u}_t + \sum_{k=1}^K w_k \varepsilon_t^{(k)}$.
- 5 Применить u_1^* к роботу, сдвинуть управляющую последовательность и перейти к следующему такту.

Алгоритм RA-MPPI. RA-MPPI (Risk-Aware MPPI) расширяет MPPI явным учётом *хвостового риска* – ожидаемых потерь в редких, но неблагоприятных сценариях исполнения управления, когда даже хорошая в среднем траектория может реализоваться существенно хуже из-за случайных отклонений [39]. Базовый MPPI рассматривает только одну стоимость на траекторию и не различает траектории с одинаковым средним, но разной устойчивостью к таким отклонениям; RA-MPPI устраняет этот недостаток, дополнительно оценивая, насколько плохо траектория ведёт себя в худших возможных реализациях.

Для количественной оценки используется мера *CVaR* (Conditional Value-at-Risk, *условная стоимость под риском*) – среднее значение стоимости среди заданного процента самых

неблагоприятных сэмпированных реализаций. Если параметр $\alpha \in [0, 1]$ задаёт уровень доверия, то CVaR_α вычисляется как среднее по худшим $(1 - \alpha) \cdot 100\%$ значениям стоимости: например, при $\alpha = 0,8$ это среднее по 20% самых худших сэмплов.

Для каждой из K номинальных траекторий разыгрывается N_d возмущённых *роллаутов* – дополнительных симуляций траектории на горизонте прогноза, в которых к управлению добавляется случайный шум $\delta \sim \mathcal{N}(0, \sigma_d^2 I)$, имитирующий неопределённость исполнения управления (по своей сути роллаут – это просто прогон управляющей последовательности через модель динамики, аналогично сэмпированию траекторий в МРПИ). По полученным N_d стоимостям вычисляется CVaR_α – среднее по худшим $\lceil (1 - \alpha)N_d \rceil$ значениям:

$$\hat{J}_k = J_k + \gamma \cdot \text{CVaR}_\alpha(\{J_k^{(n)}\}_{n=1}^{N_d}),$$

где J_k – стоимость номинального роллаута, $J_k^{(n)}$ – стоимость n -го возмущённого роллаута, $\gamma > 0$ – вес штрафа за риск. Штраф добавляется только тогда, когда CVaR превышает заданный порог C_u ; иначе $\hat{J}_k = J_k$. Затем стандартная схема взвешивания Больцмана применяется к скорректированным стоимостям $\hat{J}_k$, и RA-MРПИ сводится к классическому МРПИ, если CVaR ни для одной траектории не нарушает порог.

Пошаговая схема работы алгоритма:

- 1 Для каждой из K номинальных траекторий выполнить шаги 2(a)–2(c) МРПИ: получить J_k и сохранить последовательность возмущений $\varepsilon^{(k)}$.
- 2 Для каждой номинальной траектории k оценить хвостовой риск:
 - 2.1. разыграть N_d дополнительных возмущённых роллаутов, добавляя к управлению шум $\delta_t^{(n)} \sim \mathcal{N}(0, \sigma_d^2 I)$;
 - 2.2. вычислить стоимости $J_k^{(1)}, \dots, J_k^{(N_d)}$;
 - 2.3. отсортировать стоимости и взять среднее по худшим $\lceil (1 - \alpha)N_d \rceil$ значениям – это CVaR_α .
- 3 Скорректировать стоимость номинальной траектории: если CVaR_α превышает порог C_u , то $\hat{J}_k = J_k + \gamma \cdot \text{CVaR}_\alpha$, иначе $\hat{J}_k = J_k$.
- 4 Применить взвешивание Больцмана к скорректированным стоимостям $\hat{J}_k$ и сформировать обновлённое управление как у МРПИ.

Ключевое отличие RA-MРПИ от МРПИ – дополнительный внутренний цикл по N_d роллаутам для каждой из K номинальных траекторий, что увеличивает вычислительную стоимость в N_d раз. Взамен траектории, по которым существует существенный риск столкновения при возмущениях исполнения, получают повышенный штраф и меньший вес в итоговом управлении.

4.2 Учёт прогноза подвижных препятствий

Базовые версии MPPI и RA-MPPI на каждом шаге работают с одномоментным срезом среды: функция стоимости J_k начисляет штраф только за столкновение с клетками, занятыми в текущий момент времени. Это означает, что роллауты на горизонте T оценивают траектории относительно неизменной карты, даже если на следующем такте часть препятствий сместится. В разреженной динамике такая модель приближает реальность с систематической ошибкой: робот может выбрать траекторию, проходящую через клетку, которую препятствие займёт через несколько шагов.

Для коррекции этого эффекта предложены варианты MPPI-pred и RA-MPPI-pred, которые на каждом шаге дополнительно получают список наблюденных перемещений препятствий за прошедший такт $\mathcal{D} = \{(x_i, y_i, v_{x,i}, v_{y,i})\}$, где (x_i, y_i) — текущая позиция i -го объекта, $(v_{x,i}, v_{y,i})$ — его одношаговая скорость, вычисленная как разность последних двух наблюдаемых клеток. Внутри роллаута для каждого шага $t \in \{1, \dots, T\}$ предсказанная позиция объекта берётся как линейная экстраполяция $(\hat{x}_i^{(t)}, \hat{y}_i^{(t)}) = (x_i + tv_{x,i}, y_i + tv_{y,i})$, а к стоимости траектории добавляется мягкий штраф, активирующийся в окрестности предсказанной точки:

$$\Delta J_k^{(\text{pred})} = \sum_{t=1}^T \sum_{i \in \mathcal{D}} \frac{w_p \cdot c_{\text{obst}}}{1 + \rho_i^{(t)}} \cdot \mathbb{I}[\rho_i^{(t)} \leq 1], \quad \rho_i^{(t)} = |x_t^{(k)} - \hat{x}_i^{(t)}| + |y_t^{(k)} - \hat{y}_i^{(t)}|,$$

где $\rho_i^{(t)}$ — манхэттенское расстояние от k -й роллаут-траектории до предсказанной позиции i -го объекта на шаге t , а $w_p \in [0, 1]$ — весовой коэффициент прогнозного слагаемого. В RA-MPPI-pred прогнозный штраф применяется как к номинальному роллауту, так и к возмущённым роллаутам внутри CVaR-оценки, поэтому риск столкновения с предсказанной траекторией препятствия попадает в хвостовое распределение наравне со стохастической неопределённостью управления. Полная функция стоимости в обоих расширениях имеет вид $J_k^{(\text{pred})} = J_k + \Delta J_k^{(\text{pred})}$ для MPPI-pred и $\hat{J}_k^{(\text{pred})} = J_k^{(\text{pred})} + \gamma \cdot \text{CVaR}_\alpha(\{J_k^{(n)} + \Delta J_k^{(\text{pred},n)}\})$ для RA-MPPI-pred.

Принципиально модель прогноза остаётся максимально консервативной: она использует только один последний наблюдаемый шаг для оценки скорости, не использует поведенческой модели среды и не накладывает дополнительных ограничений на параметры базовых планировщиков. Это позволяет рассматривать MPPI-pred и RA-MPPI-pred как минимальное расширение, проверяющее ценность даже грубого линейного прогноза в задаче навигации со случайно блуждающими препятствиями.

4.3 Модель и условия эксперимента

Эксперименты выполнялись на том же бенчмарке «Arena», что и сравнение глобальных планировщиков: карта размером 49×49 клеток со статическими препятствиями и группами динамических объектов.

Реализованная программная модель является *гибридной*: карта и итоговое перемеще-

ние робота за один такт дискретны, а само сэмплирование MPPI/RA-MPPI выполняется в континуальном пространстве скоростей – то есть скорость робота при роллауте представлена двумерным вектором с вещественными компонентами $(v_x, v_y) \in \mathbb{R}^2$ и может принимать любые значения из непрерывного диапазона, а не выбирается из небольшого фиксированного набора направлений (как, например, в восьмисвязной сетке). Конкретно:

- *Карта* представлена сеточной структурой 49×49 клеток, на которой расположены статические и подвижные препятствия.
- *Состояние при роллауте* – континуальное: положение робота в процессе симуляции хранится в виде вещественных координат (x, y) , что позволяет применять стандартный аппарат MPPI с непрерывной динамикой.
- *Номинальное управление* $\bar{u}$ – двумерный вектор скорости, направленный от текущей клетки к цели и нормированный на единичную длину.
- *Возмущения* $\varepsilon_t^{(k)} \sim \mathcal{N}(0, \sigma^2 I)$ – континуальные двумерные гауссовские шумы, генерируемые из равномерных случайных чисел *преобразованием Бокса–Мюллера* – стандартным приёмом, который из пары независимых равномерных значений $u_1, u_2 \in (0, 1]$ строит пару независимых нормально распределённых значений по формулам $z_1 = \sqrt{-2 \ln u_1} \cos(2\pi u_2)$ и $z_2 = \sqrt{-2 \ln u_1} \sin(2\pi u_2)$. На каждом шаге горизонта возмущения добавляются к номинальному управлению, после чего получившаяся скорость подаётся в простейшую модель динамики $(x, y) \rightarrow (x + v_x, y + v_y)$.
- *Проверка препятствий* при роллауте дискретна: континуальные координаты на каждом шаге горизонта округляются до ближайшей клетки и сверяются со списком занятых; столкновение добавляет в стоимость траектории штраф c_{obst} .
- *Применение итогового управления* тоже дискретно: после вычисления взвешенной суммы сэмплов результирующий вектор u^* проецируется на сетку, и робот за такт переходит в одну из соседних клеток, в направлении которой u^* указывает наиболее сильно.

Такая схема позволяет использовать стандартный MPPI (с непрерывными управлениями и гауссовскими возмущениями), сохранив дискретную сеточную модель карты и единообразную дискретизацию хода робота, характерную для бенчмарков по поиску пути.

В данных экспериментах глобальные планировщики (A^*/D^* Lite) не использовались. В отличие от практической интеграции в навигационный стек Nav2 (см. главу 5), где локальный планировщик получает опорный маршрут от глобального, здесь локальные алгоритмы MPPI/RA-MPPI работают напрямую с координатами цели: номинальное управление задаётся как единичный вектор от текущей позиции к цели, без участия глобального плана. Это позволяет сравнивать локальные алгоритмы между собой в чистом виде, не смешивая их поведение с особенностями того или иного глобального планировщика.

Все четыре планировщика использовали $K = 500$ сэмплов при горизонте $T = 12$ шагов, температурный параметр $\lambda = 1,5$, стандартное отклонение шума $\sigma = 0,8$, штраф за

столкновение $c_{\text{obst}} = 200$, $w_s = 0,1$, $w_T = 15,0$. Для RA-MPPI и RA-MPPI-pred дополнительно задавались уровень доверия CVaR $\alpha = 0,8$, число возмущённых роллаутов $N_d = 20$, стандартное отклонение возмущений $\sigma_d = 0,5$, вес штрафа за риск $\gamma = 1,0$ и порог активации штрафа $C_u = 0$. Для MPPI-pred и RA-MPPI-pred вес прогнозного слагаемого был выставлен в $w_p = 0,5$, что соответствует половинному вкладу штрафа за прогнозируемое столкновение по сравнению с прямым. В качестве списка $\mathcal{D}$ на каждом шаге использовались координаты препятствий, переместившихся на предыдущем такте моделирования; неподвижные за этот такт объекты в прогноз не включались, поскольку их одношаговая скорость не определена.

Из полного набора задач отбирались 100 случайных пар «старт — цель»; ограничение на число шагов составляло 500 в статическом и 50 000 в динамическом режиме. В отличие от тестов глобальных планировщиков, количество задач было ограничено 100 ввиду большей вычислительной стоимости сэмплирующих алгоритмов на каждом шаге навигации. Аналогично предыдущим тестам, препятствия за каждый ход совершали случайное перемещение на одну клетку; тестировались три уровня загрузки: 1, 5 и 10 подвижных препятствий.

В отличие от глобальных планировщиков, для которых основным критерием являлось время перепланирования, оценка локальных методов строилась на четырёх показателях: *успешность* — доля задач, для которых найден путь до цели; *время планирования* (мс) — суммарное реальное время выполнения построения пути; *извилистость* — отношение длины пути к прямому расстоянию до цели (1,0 соответствует прямолинейному маршруту); *доля опасных сближений* — доля шагов, на которых хотя бы одна из соседних по горизонтали или вертикали клеток являлась препятствием.

4.4 Результаты экспериментов

Измерения для статического режима представлены на рисунке 4.1. На графике сопоставляются успешность, число шагов, стабильность, извилистость, время планирования и доля опасных ситуаций для всех четырёх рассматриваемых планировщиков по 100 тестовым задачам.

Измерения для динамического режима представлены на рисунке 4.2. График отражает зависимость тех же шести показателей от числа подвижных препятствий (1, 5 или 10 объектов, перемещающихся за ход).

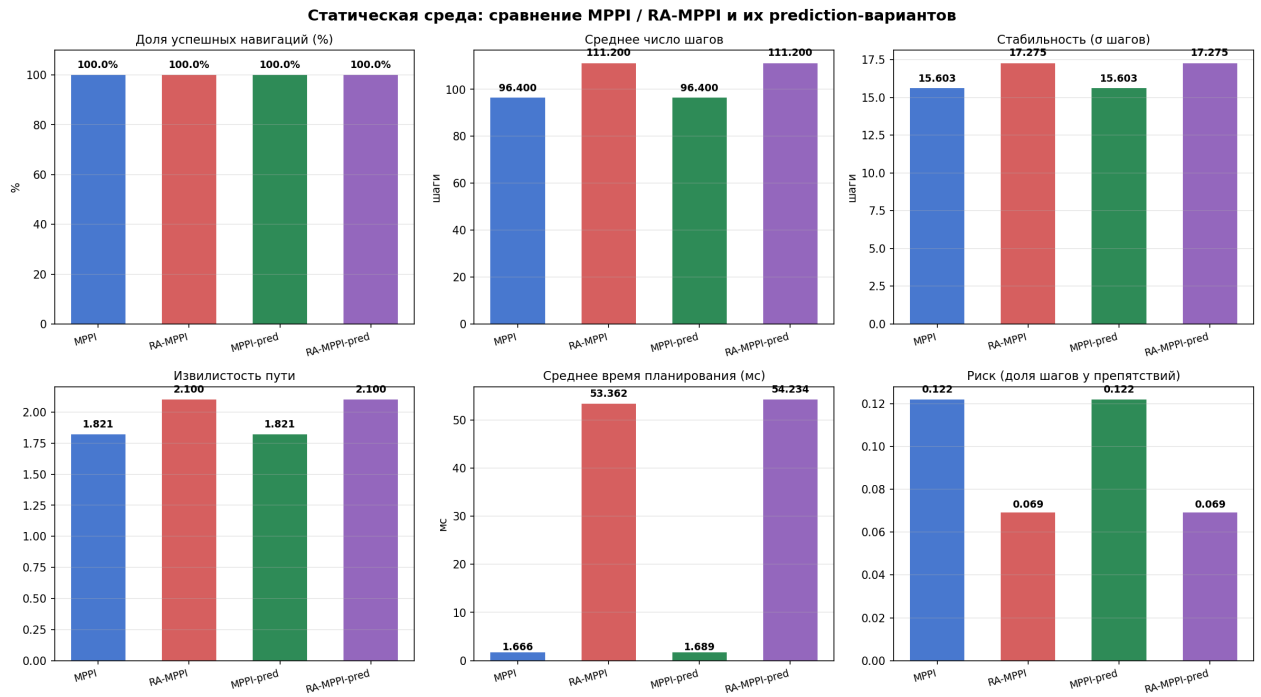


Рис. 4.1: Сравнение показателей MPPI, RA-MPPI и их prediction-вариантов в статической среде.

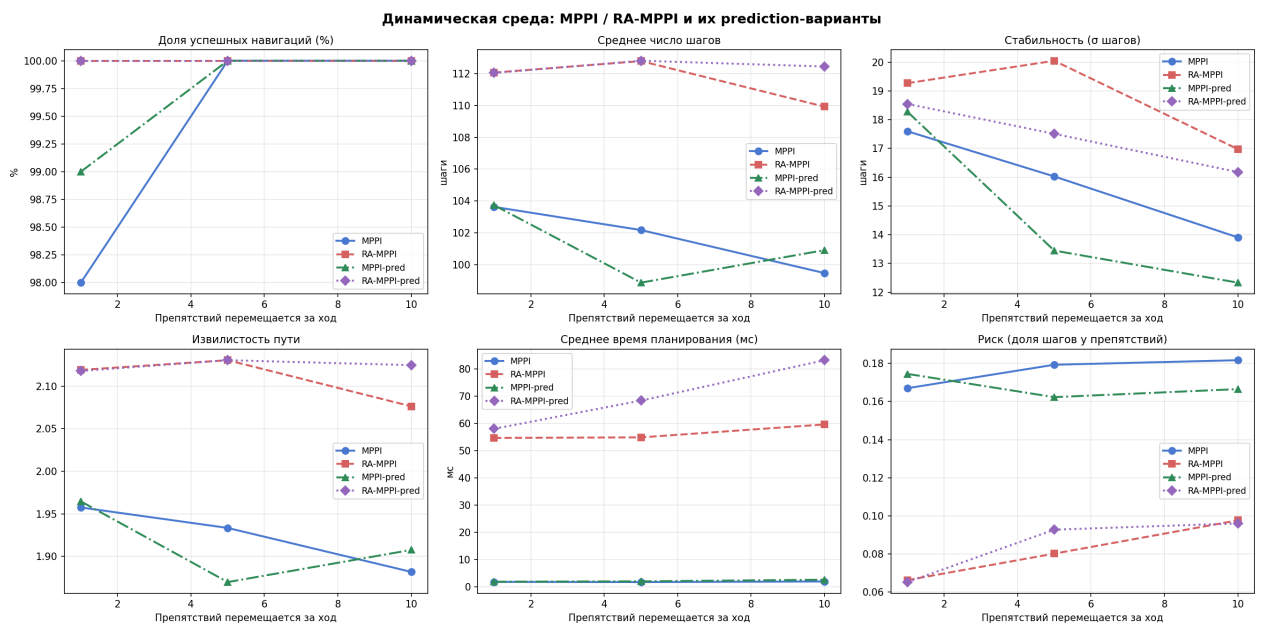


Рис. 4.2: Сравнение показателей MPPI, RA-MPPI и их prediction-вариантов в динамической среде при различном числе подвижных препятствий.

Для качественной иллюстрации различий между алгоритмами на рис. 4.3 приведены траектории робота на одном и том же тестовом случае при средней плотности динамики (5 подвижных препятствий за ход). На карте показаны статические препятствия (серые блоки), следы подвижных препятствий (красные точки с угасающей прозрачностью по времени), стартовая позиция (зелёный круг) и цель (оранжевая звезда). MPPI и RA-MPPI находят похожие по геометрии маршруты длиной около 50 шагов, тогда как RA-MPPI-pred в данном

конкретном прогоне выбирает существенно более осторожный обход и тратит 88 шагов: дополнительный штраф за предсказанные позиции подвижных объектов уводит траекторию в обход всей центральной зоны с препятствиями.

Траектории робота под четырьмя локальными планировщиками
(один и тот же тестовый случай: 5 подвижных препятствий за ход)
Зелёный круг — старт, оранжевая звезда — цель, красные точки — следы подвижных препятствий

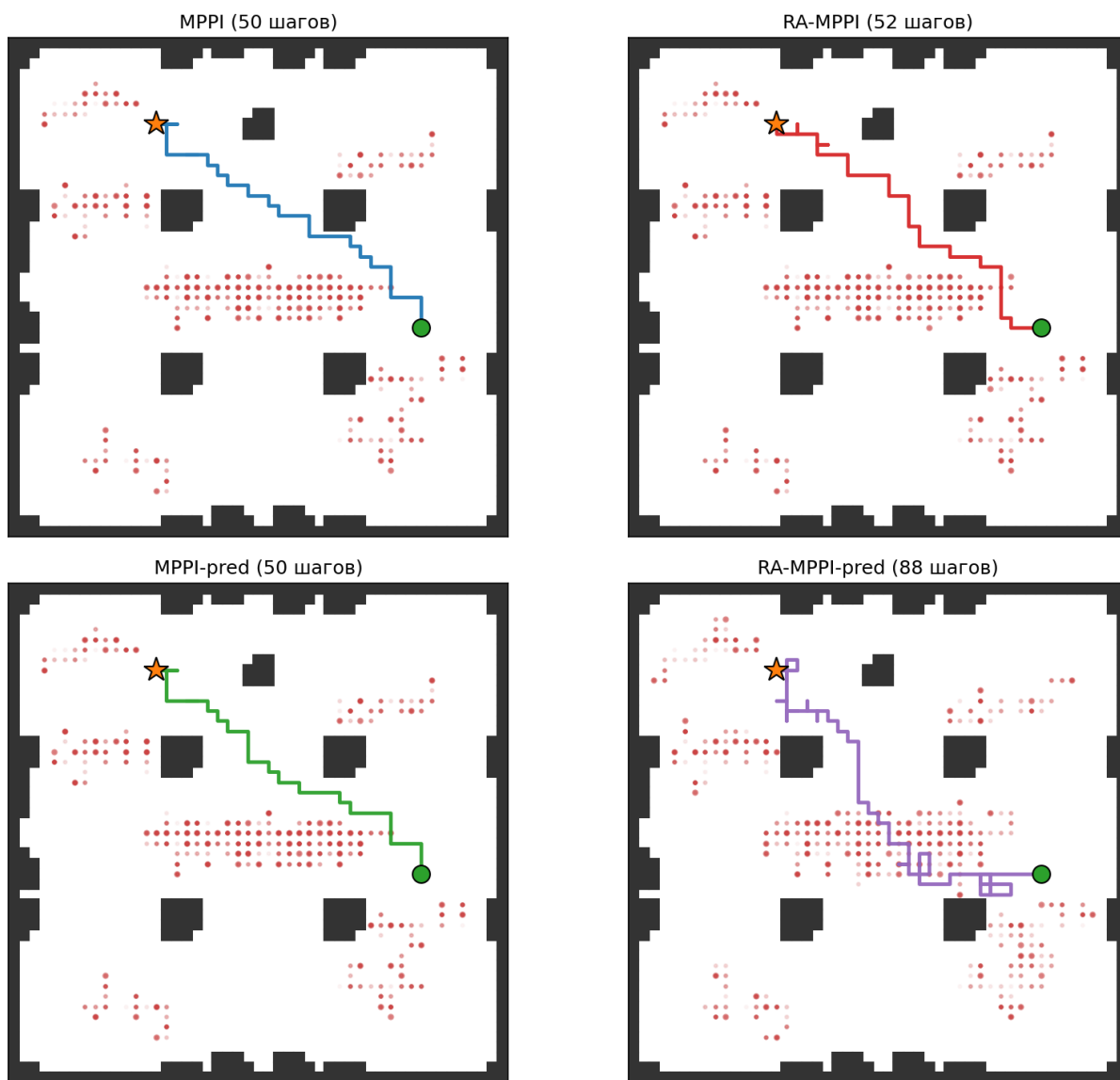


Рис. 4.3: Траектории робота под четырьмя локальными планировщиками на одном и том же тестовом случае (5 подвижных препятствий за ход): MPPI и MPPI-pred находят прямой маршрут, RA-MPPI отклоняется в сторону менее загруженной области, RA-MPPI-pred выбирает существенно более длинный обход.

4.5 Анализ производительности в статической среде

В условиях неизменной карты MPPI и MPPI-pred решили по 96 задач из 100, RA-MPPI и RA-MPPI-pred — по 94 (рис. 4.1); остальные случаи зафиксированы как таймауты по ограничению в 500 шагов на задачу. При отсутствии подвижных препятствий список $\mathcal{D}$

пуст, поэтому MPPI-pred и RA-MPPI-pred вырождаются в свои базовые версии и должны воспроизводить их метрики с точностью до издержек ветвлений; полученные измерения это подтверждают.

Для MPPI среднее время построения маршрута составило 15,07 мс, среднее число шагов — 71,5, извилистость — 2,267, доля опасных ситуаций — 12,4%. MPPI-pred показал практически идентичные показатели (15,48 мс, 71,5 шага, извилистость 2,267, доля опасных ситуаций 12,4%), причём отличие в среднем времени укладывается в шум измерения. RA-MPPI занял заметно больше вычислительного бюджета — 525,5 мс на задачу — что объясняется вычислением CVaR по $N_d = 20$ возмущённым роллаутам для каждого из $K = 500$ сэмплов; число шагов выросло до 84,8, извилистость — до 2,643. Доля опасных ситуаций при этом снизилась более чем вдвое (5,4% против 12,4% у MPPI) благодаря тому, что CVaR-штраф вытесняет рискованные сэмплы из взвешенного среднего управления. RA-MPPI-pred повторил поведение RA-MPPI с точностью до измерения (539,3 мс, 84,8 шага, извилистость 2,643, доля опасных ситуаций 5,4%).

Полученная картина согласуется с теоретическим ожиданием: учёт хвостового риска повышает осторожность планировщика и увеличивает длину пути, но снижает частоту движений вблизи препятствий. Прогнозное слагаемое в статическом режиме не активируется и не влияет ни на качество траекторий, ни на бюджет вычислений.

4.6 Анализ производительности в динамической среде

Введение движущихся препятствий заметно меняет относительное поведение алгоритмов (рис. 4.2). По всем трём уровням загрузки CVaR-варианты устойчиво превосходят базовый MPPI по доле опасных ситуаций, тогда как прогнозное слагаемое даёт наибольший вклад при умеренной плотности подвижных объектов.

При одном объекте за ход MPPI решил 97 задач из 100, MPPI-pred — 96, оба CVaR-варианта — по 100. Снижение успешности базового MPPI объясняется поведением сэмплирующего планировщика в узких проходах: единственное движущееся препятствие может одновременно занять все соседние по горизонтали и вертикали клетки робота, и взвешенная сумма управлений приводит к выходу за пределы допустимых клеток. Прогнозное слагаемое в базовом MPPI на этой плотности не даёт устойчивого выигрыша по успешности, тогда как CVaR-штраф полностью парирует остаток отказов. Доля опасных ситуаций при разреженной динамике у MPPI-pred несколько ниже, чем у MPPI (17,6% против 18,8%), что согласуется с расширением запретной зоны вокруг предсказанной позиции. У RA-MPPI и RA-MPPI-pred доля опасных ситуаций составила 6,3% и 6,8% соответственно — эффект прогноза перекрывается доминированием CVaR-штрафа и оказывается в пределах статистической погрешности.

При пяти подвижных объектах все четыре варианта сохраняют близкую к 100% успешность (RA-MPPI-pred — 99%). Длина пути и извилистость распределяются между парами «без прогноза / с прогнозом» неоднородно: у базового MPPI добавление прогноза практически не меняет геометрию траектории (64,8 против 65,6 шага, извилистость 2,036 против 2,060), а доля опасных ситуаций даже несколько возрастает (17,5% против 16,3%). У RA-

МРРІ прогноз, напротив, заметно укорачивает путь (79,6 против 83,0 шага) и снижает долю опасных сближений (5,6% против 6,4%); эффект объясняется тем, что прогнозный штраф попадает в хвост распределения CVaR и более активно влияет на отбор безопасных траекторий, чем в базовой схеме softmin.

При десяти подвижных объектах МРРІ решил 98 задач из 100, МРРІ-pred — 100, RA-МРРІ — 100, RA-МРРІ-pred — 99. Доля опасных ситуаций распределилась как 14,8%, 15,3%, 7,2% и 8,3% соответственно: переход от МРРІ к RA-МРРІ стабильно снижает долю опасных сближений приблизительно вдвое, тогда как добавление прогноза при высокой плотности динамики не даёт устойчивого выигрыша. Среднее число шагов и извилистость различаются в пределах 1–4% между парами с прогнозом и без него.

Стоимость прогноза по времени остаётся умеренной для базового МРРІ: для одного, пяти и десяти объектов МРРІ-pred требует 15,50, 13,47 и 16,07 мс соответственно против 15,62, 11,28 и 12,32 мс у МРРІ; разница укладывается в пределах $\pm 30\%$ и колеблется в обе стороны из-за вариации длины траекторий. Для RA-МРРІ-pred прогноз увеличивает время с 403,3 до 432,9 мс при $|\mathcal{D}| = 1$, с 395,0 до 473,2 мс при $|\mathcal{D}| = 5$ и с 394,0 до 553,5 мс при $|\mathcal{D}| = 10$, то есть на 7–40%. Линейный рост согласуется с тем, что прогнозное слагаемое добавляет $O(K \cdot T \cdot |\mathcal{D}|)$ операций в номинальный роллаут и $O(N_d \cdot K \cdot T \cdot |\mathcal{D}|)$ в CVaR-цикл.

4.7 Выводы по главе

Проведённое сравнение четырёх вариантов локального планировщика выявило два принципиально различных рычага улучшения навигации: учёт хвостового риска (CVaR) и явный прогноз подвижных препятствий. Их вклад оказывается разной природы и плохо взаимозаменяем: CVaR преимущественно снижает долю опасных ситуаций и повышает успешность за счёт большей длины пути и значительной вычислительной стоимости, тогда как прогноз вносит локальную поправку, наиболее заметную при умеренной плотности подвижных объектов.

Таблица 4.1: Сравнительные характеристики MPPI, RA-MPPI и их prediction-вариантов

Параметр сравнения	MPPI	RA-MPPI	MPPI-pred	RA-MPPI-pred
Статика: успешность	96%	94%	96%	94%
Статика: время, мс	15,07	525,53	15,48	539,28
Статика: число шагов	71,5	84,8	71,5	84,8
Статика: извилистость	2,267	2,643	2,267	2,643
Статика: доля опасных ситуаций	12,4%	5,4%	12,4%	5,4%
Динамика (1): успешность	97%	100%	96%	100%
Динамика (1): время, мс	15,62	403,27	15,50	432,94
Динамика (1): число шагов	88,4	84,7	84,0	85,8
Динамика (1): извилистость	2,845	2,683	2,675	2,744
Динамика (1): доля опасных ситуаций	18,8%	6,3%	17,6%	6,8%
Динамика (5): успешность	100%	100%	100%	99%
Динамика (5): время, мс	11,28	394,95	13,47	473,16
Динамика (5): число шагов	64,8	83,0	65,6	79,6
Динамика (5): извилистость	2,036	2,587	2,060	2,542
Динамика (5): доля опасных ситуаций	16,3%	6,4%	17,5%	5,6%
Динамика (10): успешность	98%	100%	100%	99%
Динамика (10): время, мс	12,32	393,99	16,07	553,48
Динамика (10): число шагов	68,3	80,3	67,1	83,1
Динамика (10): извилистость	2,135	2,505	2,109	2,632
Динамика (10): доля опасных ситуаций	14,8%	7,2%	15,3%	8,3%

Полученные данные показывают, что переход от МРРІ к RA-MРРІ снижает долю опасных ситуаций в 2–3 раза в зависимости от плотности динамики, но увеличивает время планирования примерно в 30 раз (≈ 13 мс против ≈ 430 мс) и удлиняет путь от -4% (разреженная динамика) до $+28\%$ (статика и средняя динамика). Прогнозное слагаемое, напротив, вносит более скромные изменения: для базового МРРІ оно колеблется в пределах $\pm 1,5$ процентного пункта по доле опасных ситуаций и заметного устойчивого выигрыша по геометрии траектории не показывает. Для RA-MРРІ прогноз даёт наиболее выраженный эффект при средней плотности подвижных объектов: укорачивает путь и снижает долю опасных сближений на 0,8 процентного пункта при пяти препятствиях за ход, но при разреженной и плотной динамике его вклад в долю опасных ситуаций оказывается в пределах статистической погрешности, проявляясь главным образом в виде дополнительного времени вычислений.

Таким образом, выбор конкретной комбинации остаётся компромиссом между допустимой вычислительной задержкой и требуемым уровнем безопасности. МРРІ пригоден как минимально нагрузочный вариант для разреженных сред с редко возникающими подвижными препятствиями. МРРІ-pred представляет собой малозатратное расширение, дающее выигрыш на промежуточных режимах динамики без существенного роста вычислительной стоимости. RA-MРРІ предпочтителен там, где приоритетом является интерпретируемое снижение хвостового риска, а вычислительный бюджет позволяет оплатить N_d -кратное расширение объёма роллаутов. RA-MРРІ-pred оправдан только тогда, когда от системы требуется одновременно учёт хвостового риска и явный прогноз; в исследованной случайно-блуждающей модели среды его дополнительный выигрыш по сравнению с RA-MРРІ находится на грани статистической значимости.

5 Реализация навигационного стека в ROS 2 и сквозное тестирование

В предыдущих главах исследование выполнялось в виде самостоятельных программных моделей, фокусирующихся на отдельных метриках выбранных алгоритмов: скорости перепланирования для A^*/D^* Lite и качестве траекторий для MPPI/RA-MPPI. Цель настоящей главы — интегрировать эти алгоритмы в единый навигационный стек и подтвердить их совместную работоспособность на физическом симуляторе мобильного робота.

5.1 Архитектура и используемые компоненты

Реализация выполнена на стеке ROS 2 Jazzy с физическим симулятором Gazebo Sim Harmonic (gz-sim) и фреймворком навигации Nav2. В качестве робота выбран TurtleBot3 Burger — двухколёсная дифференциальная платформа радиусом 10 см с лазерным сканером 360°, что соответствует типичной конфигурации мобильных роботов в логистике.

Архитектура построена на модели «глобальный планировщик + локальный контроллер», разделение которой обосновано в главе 2. Глобальный уровень получает на вход карту занятости и текущее положение робота и выдаёт опорный путь до цели; локальный уровень преобразует опорный путь в управляющие команды по линейной и угловой скорости, реагируя на показания сенсоров и движущиеся объекты.

Все четыре исследуемых конфигурации строятся комбинированием двух глобальных планировщиков — классического A^* , реализованного штатным модулем графового поиска пути Nav2, и собственной реализации D^* Lite, оформленной как отдельный плагин стека, — с двумя локальными контроллерами: штатным MPPI из дистрибутива Nav2 и собственной реализацией RA-MPPI, также оформленной как плагин.

Соответствие каждой пары планировщик-контроллер фиксируется отдельным конфигурационным файлом параметров Nav2. Общая структура такого файла включает узел глобального планирования, узел локального управления, узел поведенческих стратегий и исполнитель дерева поведений, который переключает робота между режимом обычной навигации и режимами восстановления; два слоя карты затрат — глобальный, отражающий известную карту окружения, и локальный, ограниченный окрестностью робота и обновляемый по показаниям сенсоров; а также стандартный набор узлов восстановления (поворот на месте, кратковременный отход назад, ожидание).

5.2 Разработанные программные пакеты

В составе работы реализованы два собственных пакета ROS 2 и набор вспомогательных скриптов для построения экспериментов и анализа bag-файлов.

5.2.1 Реализация D* Lite в виде плагина Nav2

Пакет реализует D* Lite в виде глобального планировщика-плагина Nav2 и состоит из четырёх логических частей. Первая часть – ядро алгоритма: обновление одношаговых *rhs*-оценок, очередь приоритетов с двухключевой схемой D* Lite, перерасчёт изменившихся вершин. Вторая часть отвечает за преобразование локальной карты затрат, поддерживаемой Nav2, в граф 4-связности с отсечением клеток выше порога летальной стоимости. Третья часть реализует манхэттенскую эвристику $h(s) = |s_x - g_x| + |s_y - g_y|$, допустимую при равных весах рёбер 4-связного графа. Четвёртая часть – адаптерный слой плагина: жизненный цикл внутри стека Nav2, извлечение актуальной карты затрат при каждом запросе планирования и конвертация результата во внутренний формат пути. Возникающие на диагональных участках лестничные траектории сглаживаются локальным контроллером в рамках штрафа выравнивания пути и pure-pursuit-цели без существенного замедления движения.

Внутри ядра D* Lite сохранены все инкрементальные свойства, ранее подтверждённые в главе 3. Однако в рамках Nav2-плагина каждый вызов планирования заново строит граф и запускает алгоритм от текущего состояния карты: интерфейс глобального планировщика Nav2 не передаёт сведений о том, какие именно ячейки карты изменились между вызовами, а само перепланирование инициируется самим стеком по таймеру с фиксированной частотой. В рассматриваемых сценариях это не сказывается на качестве траекторий: карта компактна (112×103 ячеек), полный поиск занимает единицы миллисекунд и сопоставим по времени с накладными расходами Nav2 на формирование результата.

5.2.2 Реализация RA-MPPI в виде плагина Nav2

Пакет реализует RA-MPPI в виде локального контроллера-плагина Nav2. Ядро алгоритма воспроизводит реализацию модельного эксперимента из главы 4: K номинальных траекторий, оценка функции стоимости, для каждой траектории — N_d возмущённых роллаутов с дополнительным шумом σ_d , вычисление CVaR_α и добавление штрафа $\gamma \cdot \text{CVaR}$, затем стандартное взвешивание Больцмана по скорректированным стоимостям.

Адаптерный слой реализует интерфейсный слой ROS 2: подписку на текущий глобальный путь, преобразование между системами координат карты и одометрии робота, формирование команды по линейной и угловой скорости и проверку достижения цели; локальный кэш карты затрат используется для быстрой проверки столкновений сэмплируемых траекторий. Прямое использование первой точки глобального плана как промежуточной цели приводило к колебаниям контроллера у криволинейных участков пути, поэтому применён классический метод pure-pursuit – стандартный приём следования по пути в робототехнике, в котором робот ориентируется не на ближайшую к нему точку плана, а на так называемую *опорную точку* (look-ahead point), расположенную впереди него на фиксированном расстоянии $L_{\text{lookahead}}$ вдоль глобального пути; именно эта опорная точка и используется в качестве цели MPPI. Значение $L_{\text{lookahead}} = 0,80$ м даёт компромисс между предсказательной способностью и агрессивностью отклонений от глобального плана.

Значения параметров пучка траекторий, использованные в главе 4 ($K = 500$, $\sigma = 0,8$), в условиях полной системы Nav2 не укладываются в целевой бюджет такта контроллера (100 мс при частоте 10 Гц): уже один проход МРРІ на $K = 500$ сэмплах при горизонте $T = 12$ занимает порядка десятков миллисекунд, а с учётом построения карты затрат, преобразований систем координат и публикации команд суммарное время выходит за рамки бюджета. В ROS-реализации число сэмплов уменьшено до $K = 150$, а стандартное отклонение шума управления увеличено до $\sigma = 1,0$, что сохраняет необходимую ширину пучка управлений при существенно меньшем числе траекторий и не приводит к преждевременной концентрации весов Больцмана на узкой группе сэмплов.

5.3 Среда тестирования и сценарии

Базовая карта — стандартный мир `turtlebot3_world`, представляющий собой квадрат $4,2 \times 4,0$ м, ограниченный стенами, с регулярной сеткой 3×3 цилиндрических колонн радиусом 15 см. Все эксперименты используют общий маршрут: старт $(-2,0, -0,5)$, цель $(1,8, 0,5)$, что соответствует диагональному пересечению карты с прохождением через верхний коридор (между северным рядом колонн и стеной). Базовая карта показана на рисунке 5.1: зелёный круг — точка старта, красный — цель.

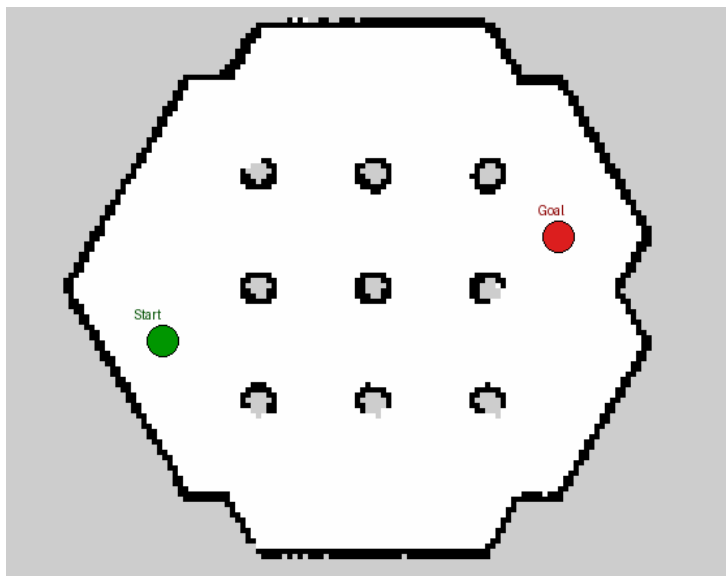


Рис. 5.1: Статическая карта `turtlebot3_world`: 3×3 колонн, старт $(-2,0; -0,5)$, цель $(1,8; 0,5)$.

Для оценки устойчивости к движущимся препятствиям сгенерированы три динамических мира: `dynamic_1` (одно препятствие, целенаправленно пересекающее траекторию), `dynamic_5` (умеренная плотность, 5 препятствий), `dynamic_10` (высокая плотность, 10 препятствий). Каждое препятствие движется кинематически по синусоиде вдоль своего коридора. В сценариях `dynamic_1` и `dynamic_5` все цилиндры используют единые параметры (период 17 с, амплитуда 0,55 м), что обеспечивает пиковую скорость 0,20 м/с. В сценарии `dynamic_10` периоды варьируются в диапазоне 12–20 с, амплитуды — в диапазоне 0,35–0,55 м, что даёт

пиковые скорости 0,13–0,21 м/с и моделирует более разнородную динамическую среду. Указанный диапазон скоростей сопоставим с целевой скоростью робота ($\sim 0,25$ м/с) и позволяет локальному контроллеру в принципе предсказать траекторию препятствия на горизонте планирования. Расположение цилиндров и их коридоры движения для трёх плотностей показаны на рисунке 5.2: синие круги — начальные позиции, светло-синие линии — допустимые отрезки синусоидального движения.

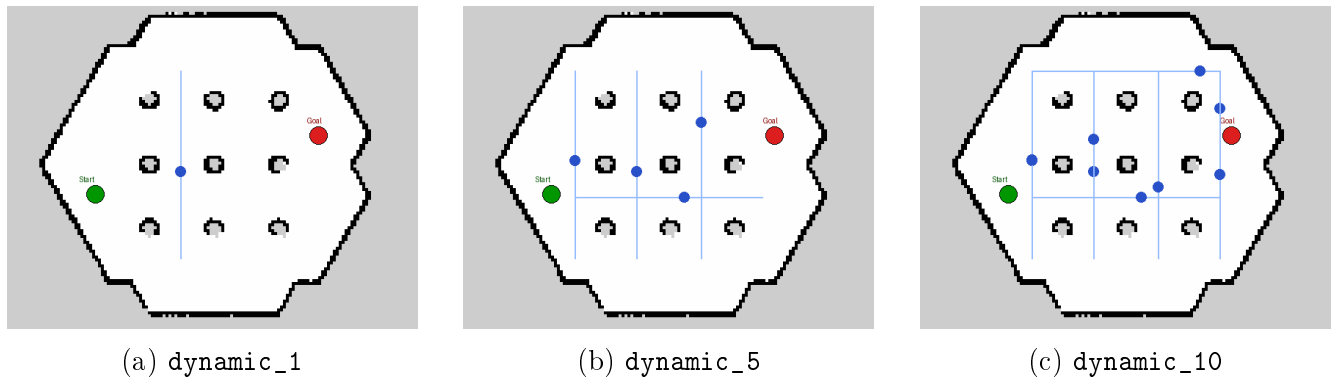


Рис. 5.2: Карты с подвижными цилиндрическими препятствиями.

На каждом сценарии фиксируются пять метрик: *успешность* — достиг ли робот окрестности цели (толерантность 0,35 м) в пределах 120-секундного бюджета; *итоговое отклонение* — расстояние от финальной точки робота до цели; *длина пути* — суммарное смещение по одометрии; *длительность* — время от первого до последнего сообщения о позе робота; *опасные сближения* — число эпизодов, когда штатный модуль контроля безопасности Nav2 регистрировал критически малую дистанцию между роботом и ближайшим препятствием (как статическим, так и подвижным) и принудительно тормозил или останавливал робот; последовательные срабатывания, разделённые паузой менее 2 с, объединяются в одно событие, чтобы не учитывать дважды одну и ту же опасную ситуацию.

Подвижные препятствия перемещаются по карте кинематически (без физического моделирования контактов): каждый цилиндр имеет заранее назначенную синусоидальную траекторию вдоль своего коридора и обновляет позицию с частотой 10 Гц.

Логика движения дополнена поведенческим условием: цилиндр не вытесняет робот. Запланированная позиция вычисляется заранее, и если она попадает в зону безопасности радиуса 0,30 м вокруг робота, цилиндр временно удерживается на текущей координате и не смещается в сторону робота. Такое поведение сохраняет реалистичность твёрдого неподвижного препятствия и требует от локального контроллера активного объезда.

5.4 Линейное предсказание траекторий динамических препятствий

Штатный MPPI-контроллер из дистрибутива Nav2 не различает статические и динамические препятствия: стоимость роллаута определяется по текущей карте затрат, которая обновляется лишь на следующем цикле Nav2. Для конфигураций с собственным контроллером RA-MPPI (пары A^* +RA-MPPI и D^* Lite+RA-MPPI) реализовано линейное предсказание

положений цилиндров непосредственно внутри оценщика стоимости роллаутов. Конфигурации со штатным МРРІ-контроллером предсказания не выполняют и воспринимают движущиеся цилиндры как статические объекты, сместившиеся к текущему обновлению карты затрат.

Передача состояний цилиндров между процессом перемещения цилиндров и контроллером реализована через общий файл, обновляемый на каждом такте (10 Гц): для каждого цилиндра записывается его текущая позиция (x, y) и аналитическая скорость (v_x, v_y) . Скорость вычисляется как производная заданной синусоиды: $v = A \cdot \frac{2\pi}{T} \cos(\frac{2\pi}{T}t + \varphi)$. Контроллер RA-MРРІ перед каждым запросом очередного шага считывает этот файл и переводит мировые координаты в координаты ячейки карты затрат стандартными средствами Nav2:

$$row = \frac{y_w - o_y}{\Delta}, \quad col = \frac{x_w - o_x}{\Delta}, \quad v_{row} = \frac{v_y}{\Delta}, \quad v_{col} = \frac{v_x}{\Delta},$$

где Δ — разрешение карты затрат (м/ячейку), (o_x, o_y) — координаты начала карты в мировой системе координат.

В каждом шаге роллаута t вычисляется предсказанное положение препятствия:

$$p_{row}(t) = row + v_{row} \cdot t \cdot \Delta t, \quad p_{col}(t) = col + v_{col} \cdot t \cdot \Delta t,$$

и к стоимости роллаута добавляется штраф:

$$c_{obs}(t) = \begin{cases} c_{hard}, & d(t) < r_{hard}, \\ c_{soft} (1 - d(t)/r_{soft})^2, & r_{hard} \leq d(t) < r_{soft}, \\ 0, & \text{иначе,} \end{cases}$$

где $d(t)$ — расстояние от позиции роллаута до предсказанного центра цилиндра. В RA-MРРІ тот же штраф применяется в обоих типах роллаутов: номинальных (для оценки средней стоимости) и возмущённых (для вычисления CVaR-хвоста), что позволяет оценивать риск столкновения с движущимся препятствием с учётом неопределённости управления.

Параметры штрафа: жёсткий радиус $r_{hard} = (r_{cyl} + m_{hard})/\Delta$ ($r_{cyl} = 0,15$ м — радиус цилиндра, $m_{hard} = 0,09$ м — запас), мягкий $r_{soft} = 0,70$ м/ Δ .

5.5 Результаты экспериментов

Тестирование выполнено на 16 сценариях — декартово произведение четырёх комбинаций *планировщик-контроллер* и четырёх миров. Каждый сценарий запускался 10 раз независимо; на каждом прогоне поднимался независимый экземпляр Gazebo, Nav2 и процесса перемещения цилиндров, bag-файл записывался полностью и анализировался Python-скриптом пост-обработки. Фиксировалось число успешных и неудачных попыток, а метрические показатели усреднялись по успешным прогонам.

Сводные результаты приведены в таблице 5.1. Колонка *Планировщик* обозначает ком-

бинацию глобального планировщика и локального контроллера (один из A^* / D^* Lite в паре с одним из MPPI / RA-MPPI); колонка *Мир* — тестовый сценарий: *static* (без подвижных препятствий) или *dynamic_N* (мир с N движущимися цилиндрами). Колонки *Усп.* и *Неуд.* содержат число успешных и неудачных прогонов из десяти; успешным считается прогон, в котором робот достиг окрестности цели с толерантностью 0,35 м за 120 с. Колонка *Откл.*, м — среднее расстояние от финальной точки робота до цели по успешным прогонам; *Путь*, м — средняя суммарная длина траектории по данным одометрии; *Время*, с — средняя длительность теста от первой до последней опубликованной позы робота. Колонка *Сбл.* приводит среднее число срабатываний штатного модуля контроля безопасности Nav2, разделённых паузой более 2 с, и отражает частоту принудительных торможений из-за динамических препятствий.

Таблица 5.1: Результаты сквозного тестирования навигационного стека ROS 2 / Nav2 (10 прогонов на сценарий).

Планировщик	Мир	Усп.	Неуд.	Откл., м	Путь, м	Время, с	Сбл.
A^* + MPPI	static	10	0	0,252	4,21	18,1	0,00
D^* Lite + MPPI	static	10	0	0,254	4,46	21,4	0,00
A^* + RA-MPPI	static	10	0	0,190	4,47	18,7	0,00
D^* Lite + RA-MPPI	static	10	0	0,245	4,45	16,6	0,00
A^* + MPPI	dynamic_1	10	0	0,240	4,25	19,3	0,00
D^* Lite + MPPI	dynamic_1	10	0	0,262	4,47	21,8	0,10
A^* + RA-MPPI	dynamic_1	10	0	0,180	4,62	22,6	0,00
D^* Lite + RA-MPPI	dynamic_1	10	0	0,237	4,49	18,0	0,00
A^* + MPPI	dynamic_5	9	1	0,238	4,78	30,7	0,30
D^* Lite + MPPI	dynamic_5	10	0	0,254	4,51	22,1	0,00
A^* + RA-MPPI	dynamic_5	10	0	0,194	4,91	22,2	0,00
D^* Lite + RA-MPPI	dynamic_5	10	0	0,221	4,58	19,4	0,00
A^* + MPPI	dynamic_10	10	0	0,221	4,84	41,9	1,00
D^* Lite + MPPI	dynamic_10	10	0	0,258	4,55	24,3	0,00
A^* + RA-MPPI	dynamic_10	10	0	0,148	5,26	27,3	0,20
D^* Lite + RA-MPPI	dynamic_10	9	1	0,209	4,90	23,6	0,22

В статическом мире все четыре конфигурации справляются со 100 % успешностью. Быстрейшим оказывается D^* Lite + RA-MPPI (16,6 с), A^* + MPPI и A^* + RA-MPPI сопоставимы (18,1 и 18,7 с), тогда как D^* Lite + MPPI немного медленнее (21,4 с). Длина пути варьируется в узком диапазоне 4,2–4,5 м, что подтверждает малозначимость выбора глобального планировщика для метрики пути в простых условиях.

При наличии одного движущегося цилиндра (*dynamic_1*) все конфигурации также достигают 100 % успешности; порядок скоростей сохраняется: D^* Lite + RA-MPPI лидирует (18,0 с), A^* + MPPI и A^* + RA-MPPI — 19,3 и 22,6 с, D^* Lite + MPPI — 21,8 с. Единичное срабатывание модуля контроля безопасности у D^* Lite + MPPI (0,10) отражает то, что инкрементальный планировщик реже уводит робота в обход, вынуждая локальный контроллер тормозить.

В сценарии *dynamic_5* впервые проявляется нестабильность A^* + MPPI: 9 успешных прогонов из 10, среднее время на успешных прогонах — 30,7 с, среднее число сближений — 0,30. Остальные три конфигурации сохраняют 100 % успешность при времени 19–22 с.

Неудача $A^* + \text{MPPI}$ воспроизводится стабильно (подтверждена повторными прогонами): при плотном расположении пяти цилиндров MPPI-контроллер с эвристикой A^* периодически попадает в тупиковое локальное оптимальное состояние, из которого не может выйти за 120 с.

В наиболее сложном сценарии `dynamic_10` контраст между конфигурациями максимален. $A^* + \text{MPPI}$ остаётся полностью успешным (10/10), однако ценой резкого роста времени (41,9 с) и высокого числа сближений (1,00 за прогон): контроллер многократно тормозит из-за движущихся цилиндров. $D^* \text{ Lite} + \text{MPPI}$ справляется значительно быстрее (24,3 с) при нулевых сближениях — инкрементальное перепланирование позволяет заблаговременно обойти динамические объекты. $A^* + \text{RA-MPPI}$ показывает 27,3 с при 0,20 сближениях: CVaR-штраф удерживает робота от опасной близости, однако без глобального перепланирования траектория длиннее. $D^* \text{ Lite} + \text{RA-MPPI}$ даёт 9/10 (23,6 с, 0,22 сближения): один неудачный прогон вызван блокировкой сразу несколькими цилиндрами, после чего навигация не укладывалась в таймаут.

Финальное отклонение от цели по успешным прогонам составляет 0,15–0,26 м во всех сценариях, что существенно меньше порога 0,35 м и свидетельствует о стабильной точности позиционирования независимо от сложности среды.

5.6 Выводы по главе

Эксперименты показали, что исследованные в главах 3–4 алгоритмы успешно работают в составе целевой инженерной системы — ROS 2 / Nav2 / Gazebo Sim. Сборка и стабилизация интегрированного стека потребовала разработки двух собственных пакетов (`dstar_lite_planner`, `ramppi_controller`), кинематического модуля перемещения препятствий и набора скриптов оркестрации тестов и пост-обработки. Из 16 сценариев 14 завершились со 100 % успешностью; единственные исключения — $A^* + \text{MPPI}$ в `dynamic_5` (9/10) и $D^* \text{ Lite} + \text{RA-MPPI}$ в `dynamic_10` (9/10) — отражают реальную сложность навигации при высокой плотности движущихся препятствий, а не дефекты реализации. Наиболее стабильной и быстрой во всех динамических мирах оказывается связка $D^* \text{ Lite} + \text{MPPI}$, сочетающая инкрементальное перепланирование с нулевыми сближениями при `dynamic_10`; $D^* \text{ Lite} + \text{RA-MPPI}$ демонстрирует наименьшее время в статичных и малодинамичных сценариях (16,6 и 18,0 с).

Описание применения генеративной модели

При выполнении работы в качестве вспомогательного инструмента применялась генеративная модель Claude Opus 4.8 (компания Anthropic). Доступ к модели осуществлялся через её официальный сайт <https://claude.ai> и интерфейс командной строки Claude Code.

Целями применения модели: редактирование текста работы и тестирование разработанного программного кода. При редактировании текста модель использовалась для устранения стилистических и грамматических неточностей и повышения связности изложения. При тестировании программного кода модель привлекалась для проверки работоспособности реализованных компонентов и выявления ошибок в ходе запуска тестов.

По способу применения взаимодействие велось в интерактивном диалоговом режиме: описывалась задача и место, которое требовалось проверить, после чего модель предлагала правки; предложенные правки изучались, проверялись вручную и на их основе редактировался текст или код.

Заключение

В рамках курсовой работы решена комплексная задача исследования и практической реализации алгоритмов навигации мобильного робота в статической и динамической среде.

В аналитической части выполнен систематический обзор современных методов глобального и локального планирования. Рассмотрены графовые, сэмплирующие и бионические глобальные планировщики, а также классические контроллеры, геометрические и оптимизационные методы локального управления. На основе сравнительного анализа обоснован выбор D^* Lite в качестве глобального планировщика, обеспечивающего инкрементальное перепланирование при локальных изменениях карты. На уровне локального управления для последующего экспериментального сравнения отобраны MPPI и RA-MPPI — стохастические предсказательные контроллеры, способные работать с нелинейной динамикой и учитывать препятствия через функцию стоимости.

Для экспериментального сравнения глобальных планировщиков разработана собственная программная модель на C++ и проведено сравнение A^* и D^* Lite на модифицированной карте «Арена» (49×49 клеток) при 1000 случайных задачах на каждую конфигурацию. Показано, что в статической среде A^* остаётся оптимальным выбором благодаря лёгкому однократному поиску (0,22–0,74 мс против 1,26–5,69 мс у D^* Lite), однако в режиме высокой динамики (15 препятствий за ход) D^* Lite сокращает время обновления пути в 12–15 раз (~ 1 –9 мс против ~ 2 –156 мс), что делает его единственным масштабируемым решением для систем реального времени.

На той же модели проведено сравнение локальных контроллеров MPPI и RA-MPPI на 100 навигационных задачах. RA-MPPI реализует мягкий CVaR-штраф на основе возмущённых роллаутов и устойчиво превосходит MPPI по доле опасных сближений с препятствиями (в несколько раз ниже во всех режимах динамики), а также показывает несколько более высокую успешность в плотных сценариях. Платой за это становится значительный рост вычислительных затрат, обусловленный дополнительным внутренним циклом по возмущённым роллаутам в CVaR-оценке.

На этапе практической реализации алгоритмы интегрированы в навигационный стек ROS 2 / Nav2 / Gazebo Sim в виде двух собственных плагинов: глобального планировщика D^* Lite и локального контроллера RA-MPPI. Дополнительно разработаны модуль кинематического перемещения цилиндров и набор скриптов оркестрации экспериментов. На сквозном бенчмарке из 16 сценариев (4 пары планировщик–контроллер на 4 мирах с числом подвижных препятствий 0–10, по 10 прогонов на каждый сценарий) подтверждена совместная работоспособность всех конфигураций на физическом симуляторе TurtleBot3 Burger; 14 из 16 сценариев достигли 100 % успешности.

Полученные результаты подтверждают реализуемость гибридной архитектуры на собственных алгоритмах в составе индустриального стека Nav2 и могут служить основой для дальнейших исследований автономной навигации в непредсказуемой динамической среде и многороботных системах.

Список литературы

- [1] Kuanqi CAI, Chaoqun WANG, Jiyu CHENG, Shuang SONIG, SILVA Clarence W. DE и MENG Max Q.-H. «Mobile Robot Path Planning in Dynamic Environments: A Survey.» В: *Instrumentation* 6.2 (2024), с. 90—100.
- [2] Karthik Karur, Nitin Sharma, Chinmay Dharmatti и Joshua E. Siegel. «A Survey of Path Planning Algorithms for Mobile Robots». В: *Vehicles* 3.3 (2021), с. 448—468.
- [3] Chengmin Zhou, Bingding Huang и Pasi Fränti. «A review of motion planning algorithms for intelligent robotics». В: *arXiv preprint, arXiv:2102.02376, version 2* (2021).
- [4] E. W. Dijkstra. «A Note on Two Problems in Connexion with Graphs». В: *Numerische Mathematik* 1 (1959), с. 269—271.
- [5] Peter Hart, Nils Nilsson и Bertram Raphael. «A Formal Basis for the Heuristic Determination of Minimum Cost Paths». В: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), с. 100—107.
- [6] Anthony Stentz. «Optimal and Efficient Path Planning for Partially-Known Environments». В: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1994, с. 3310—3317.
- [7] Sven Koenig, Maxim Likhachev и David Furcy. «Lifelong Planning A*». В: *Artificial Intelligence* 155.1–2 (2004), с. 93—146.
- [8] Sven Koenig и Maxim Likhachev. «D* Lite». В: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)* (2002), с. 476—483.
- [9] Maxim Likhachev, Geoffrey J. Gordon и Sebastian Thrun. «ARA*: Anytime A* with Provable Bounds on Sub-Optimality». В: *Advances in Neural Information Processing Systems (NIPS)*. 2003, с. 767—774.
- [10] Liding Zhang, Kuanqi Cai, Zewei Sun, Zhenshan Bing, Chaoqun Wang, Luis Figueredo, Sami Haddadin и Alois Knoll. «Motion planning for robotics: A review for sampling-based planners». В: ().
- [11] Steven M. LaValle. «Rapidly-exploring Random Trees: A New Tool for Path Planning». В: *TR-98-11, Computer Science Dept., Iowa State University* (1998).
- [12] Sertac Karaman и Emilio Frazzoli. «Sampling-based Algorithms for Optimal Motion Planning». В: *The International Journal of Robotics Research* 30.7 (2011), с. 846—894.
- [13] Jonathan D. Gammell, Siddhartha S. Srinivasa и Timothy D. Barfoot. «Batch Informed Trees (BIT*): Sampling-based Optimal Planning via the Heuristically Guided Search of Implicit Random Geometric Graphs». В: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (2015), с. 3067—3074.

- [14] Sanjiban Choudhury, Jonathan D. Gammell, Timothy D. Barfoot, Siddhartha S. Srinivasa и Sebastian Scherer. «Regionally Accelerated Batch Informed Trees (RABIT*): A Framework to Integrate Local Information into Optimal Path Planning». B: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (2016), с. 4207—4214.
- [15] Wenzheng Chi, Chaoqun Wang, Jiankun Wang и Max Q.-H. Meng. «Risk-DTRRT-Based Optimal Motion Planning Algorithm for Mobile Robots». B: *IEEE Transactions on Automation Science and Engineering* 16.3 (2018/2019), с. 1271—1288.
- [16] Shiwei Lin, Jianguo Wang и Xiaoying Kong. «Bio-Inspired Reactive Approaches for Automated Guided Vehicle Path Planning: A Review». B: *Biomimetics* 11.1 (2026).
- [17] J. H. Holland. *Adaptation in Natural and Artificial Systems*. Four volumes. University of Michigan Press, 1975.
- [18] Dorigo M., Maniezzo V. и Colnari A. «Ant System: Optimization by a Colony of Cooperating Agents». B: *IEEE Transactions on Systems, Man, and Cybernetics – Part B* 26.1 (1996), с. 29—41.
- [19] Karaboga D. и Basturk B. «A powerful and efficient algorithm for numerical function optimization: Artificial Bee Colony (ABC) algorithm». B: *Journal of Global Optimization* 39.3 (2007), с. 459—471.
- [20] Kennedy J. и Eberhart R. «Particle Swarm Optimization». B: *Proceedings of IEEE International Conference on Neural Networks (ICNN'95)* (1995), с. 1942—1948.
- [21] Alexandru Bârsan. «Position Control of a Mobile Robot through PID Controller». B: *Acta Universitatis Cibiniensis. Technical Series* 71 (2019), с. 14—20.
- [22] Zefan Su, Hanchen Yao, Jianwei Peng, Zhelin Liao, Zengwei Wang, Hui Yu, Houde Dai и Tim C. Lueth. «LQR-based control strategy for improving human–robot companionship and natural obstacle avoidance». B: *Biomimetic Intelligence and Robotics* 4.4 (2024).
- [23] Baiyu Tian, Tianliang Lin, Chunhui Zhang, Zhongshen Li, Shengjie Fu и Qihuai Chen. «Mixed Linear Quadratic Regulator Controller Design for Path Tracking Control of Autonomous Tracked Vehicles». B: *Lecture Notes in Mechanical Engineering* (2024), с. 127—136.
- [24] Chukwuma Samuel Okafor, Chimaihe B. Mbachu, Chidiebere N. Muoghalu и Bonaventure O. Ekengwu. «Positioning and Trajectory Tracking with Deflection Suppression in Flexible Link Robotic Manipulator Using PID-LQR Controller». B: *Asian Journal of Science, Technology, Engineering, and Art* 3.4 (2025), с. 1131—1146.
- [25] Dieter Fox, Wolfram Burgard и Sebastian Thrun. «The Dynamic Window Approach to Collision Avoidance». B: *IEEE Robotics & Automation Magazine* (1997).
- [26] Yanjie Cao и Norzalilah Mohamad Nor. «An improved dynamic window approach algorithm for dynamic obstacle avoidance in mobile robot formation». B: *Decision Analytics Journal* (2024).

- [27] Paolo Fiorini и Zvi Shiller. «Motion Planning in Dynamic Environments Using Velocity Obstacles». В: *The International Journal of Robotics Research* 17.7 (1998), с. 760—772.
- [28] Wanxin Liu, Bo Zhang, Pudong Liu, Jian Pan и Shiyu Chen. «Velocity Obstacle Guided Motion Planning Method in Dynamic Environments». В: *Journal of King Saud University – Computer and Information Sciences* 36.1 (2024).
- [29] Oussama Khatib. «Real-Time Obstacle Avoidance for Manipulators and Mobile Robots». В: *The International Journal of Robotics Research* 5.1 (1986), с. 90—98.
- [30] Shahid Mohammad Mulla, Aryan Kanakapudi, Lakshmi Narasimhan и Anuj Tiwari. «RF-Source Seeking with Obstacle Avoidance using Real-time Modified Artificial Potential Fields in Unknown Environments». В: *arXiv preprint, arXiv:2506.06811, version 1* (2025).
- [31] Johann Borenstein и Yoram Koren. «The Vector Field Histogram — Fast Obstacle Avoidance for Mobile Robots». В: *IEEE Journal of Robotics and Automation* 7.3 (1991).
- [32] Andrej Babinec, Martin Dekan, František Duchoň и Anton Vitko. «Modifications of VFH Navigation Methods for Mobile Robots». В: *Procedia Engineering* 48 (2012), с. 10—14.
- [33] Shuyou Yu, Matthias Hirche, Yanjun Huang, Hong Chen и Frank Allgöwer. «Model predictive control for autonomous ground vehicles: a review». В: *Autonomous Intelligent Systems* 1.4 (2021).
- [34] Williams G., Aldrich A. и Theodorou E. A. «Model Predictive Path Integral Control: From Theory to Parallel Computation». В: *Journal of Guidance, Control, and Dynamics* 40.2 (2017), с. 344—357.
- [35] Williams G., Drews P., Goldfain B., Rehg J. M. и Theodorou E. A. «Aggressive driving with model predictive path integral control». В: *IEEE International Conference on Robotics and Automation (ICRA)* (2016), с. 1433—1440.
- [36] Guzman R., Oliveira R. и Ramos F. «Model predictive path integral control for car driving with autogenerated cost map based on prior map and camera image». В: *2019 IEEE Intelligent Transportation Systems Conference (ITSC)* (2019), с. 2109—2114.
- [37] Isin M. Balci, Efstathios Bakolas, Bogdan I. Vlahov и Evangelos A. Theodorou. «Constrained Covariance Steering Based Tube-MPPI». В: *2022 American Control Conference (ACC)* (2022), с. 4197—4202.
- [38] Gandhi M. S., Vlahov B., Gibson J., Williams G. и Theodorou E. A. «Robust Model Predictive Path Integral Control: Analysis and Performance Guarantees». В: *IEEE Robotics and Automation Letters* 6.2 (2021), с. 1423—1430.
- [39] Ji Yin, Zheng Zhang и Panagiotis Tsiotras. «Risk-Aware Model Predictive Path Integral Control Using Conditional Value-at-Risk». В: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2023, с. 7937—7943.

- [40] Guzman R., Oliveira R. и Ramos F. «Adaptive Model Predictive Control by Learning Classifiers». В: *Proceedings of The 4th Annual Learning for Dynamics and Control Conference (PMLR)* 168 (2022), с. 480—491.
- [41] Nathan R. Sturtevant. «Benchmarks for Grid-Based Pathfinding». В: *IEEE Transactions on Computational Intelligence and AI in Games* 4.2 (2012). URL: <https://www.movingai.com/benchmarks/> (дата обр. 2026-06-01), с. 144—148.